

Advanced Natural Language Processing

Discussion 5: Towards Transformers

Department of Advanced Computer Sciences (DACS)
Master of AI / DSDM
Prof dr ir Jan C. Scholtes – Course year 2026-2027

Discussion Lecture: Transformers

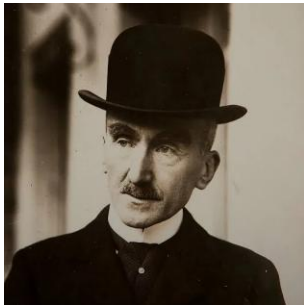


Maastricht University

Any Questions on the Lecture on MHSA and Transformer Architecture

How Humans Experience Time — Before LSTMs & Attention

- We assume memory works like a filmstrip: fixed-order moments, each waiting for the last — but does lived experience actually feel that way?
- Anticipating tomorrow can shape how we experience today; a smell or song can pull a memory from decades ago straight into the present, skipping everything in between.
- French philosopher Henri Bergson argued that clock time, a uniform line of identical ticks, is a convenient fiction; real duration is a blend where past, present, and anticipated future interpenetrate rather than queue up.
- Attention is the mechanism that lets human cognition break free of strict chronological order
- Early sequence models (RNNs/LSTMs) built the filmstrip assumption directly into their architecture: process token t only after token $t-1$ finishes.



If even human cognition, per Bergson, doesn't run on a strict sequential queue, why should language models?

Attention is the associative, relevance-driven spotlight that lets human cognition retrieve any past moment or project any future one on demand, indexed by meaning rather than timestamp

Parallelization in LSTM Models

LSTM's hidden state h_t depends on $h_{\{t-1\}}$, so step 50 can't start before step 49 finishes.

Where parallelism is possible:

- the four gates within one timestep are independent matrix multiplies (parallelizable), and different sequences in a training batch don't depend on each other either.
- Stacked layers can even pipeline, layer 2 can start on earlier timesteps while layer 1 churns through later ones.

Where it is not possible: none of that touches the one axis that matters most for long sequences, parallelizing across time, within a single sequence. That requires the recurrence itself to break, which only happens if you strip out the nonlinearity ($h_t = A \cdot h_{\{t-1\}} + B \cdot x_t$), turning it into a single matrix formula with no loop, exactly the trick behind State Space Models and linear RNNs.

The real bottleneck, concretely: to compute h_{100} you must first have h_{99} , which needs h_{98} , ... all the way back to h_1 , a chain of 100 sequentially dependent GPU kernel launches that can't be batched into one, even though each individual step is cheap.

On a GPU built for throughput via massive parallel work, this is like running 100 people through a single-file door one at a time instead of letting them all walk through a wide-open hallway together — the work per step is trivial, but the waiting dominates.

Dropping infinite recursion for Parallel Attention (MHSA)

- RNNs processed tokens one at a time, like recursion unwinding a stack. Slow, but able to run arbitrarily deep, closer to Chomsky's context-free grammars.
- Transformers dropped recurrence for MHSA, letting GPUs process whole sequences in parallel. Speed went up enormously.
- The cost: depth is fixed by layer count, not input length. This bounds Transformers to TC^0 , closer to a finite-state machine, so they can fail at tasks needing unbounded recursion (bracket matching, graph traversal) once inputs exceed their fixed depth.
- Chain-of-thought patches this: writing out steps token by token re-injects sequential depth, simulating the recursion RNNs had built in.

Transformers traded unbounded recursive depth for parallel speed. Chain-of-thought manually restores some of that depth, one token at a time.

2017: “Attention is All you need”

WIRED



BY STEVEN LEVY THE BIG STORY MAR 28, 2024 6:00 AM

8 Google Employees Invented Modern AI. Here's the Inside Story

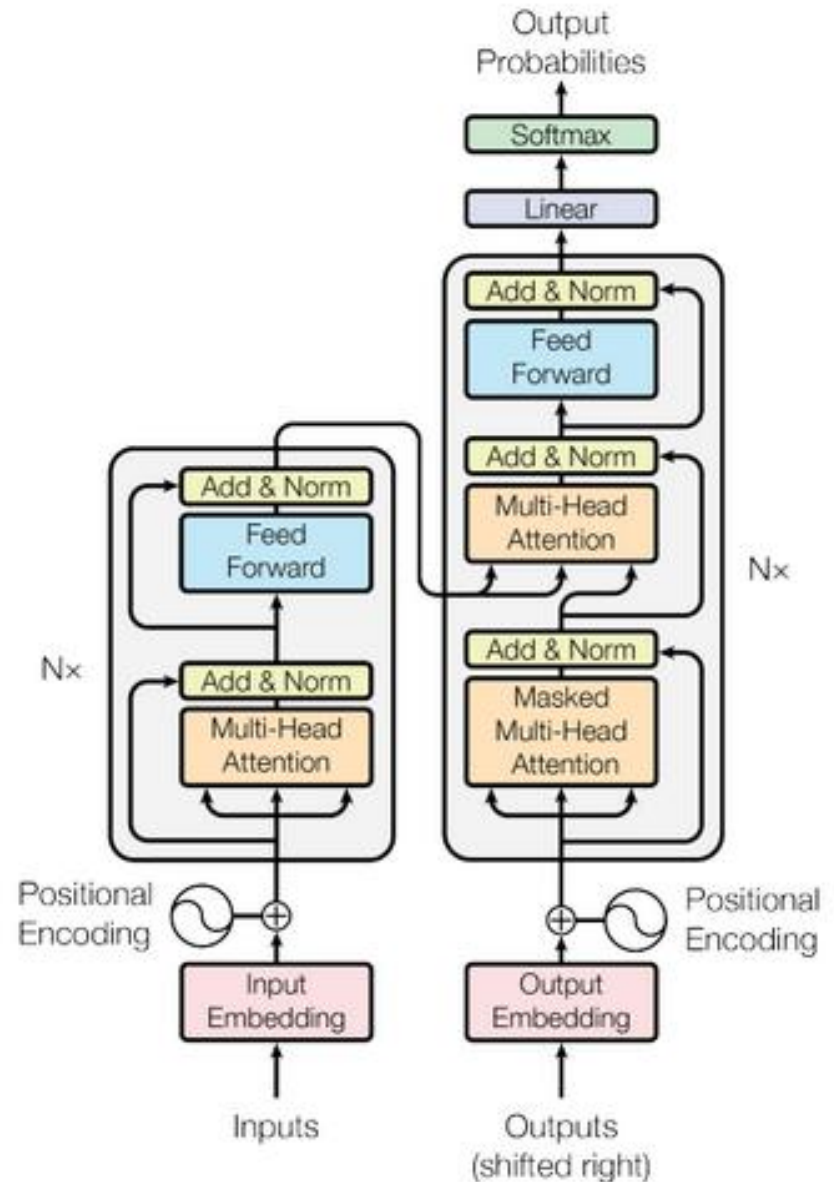
They met by chance, got hooked on an idea, and wrote the “Transformers” paper—the most consequential tech breakthrough in recent history.

The paper "Attention Is All You Need" introduced the Transformer architecture, which revolutionized natural language processing by using self-attention mechanisms to achieve state-of-the-art performance without relying on recurrent or convolutional networks.

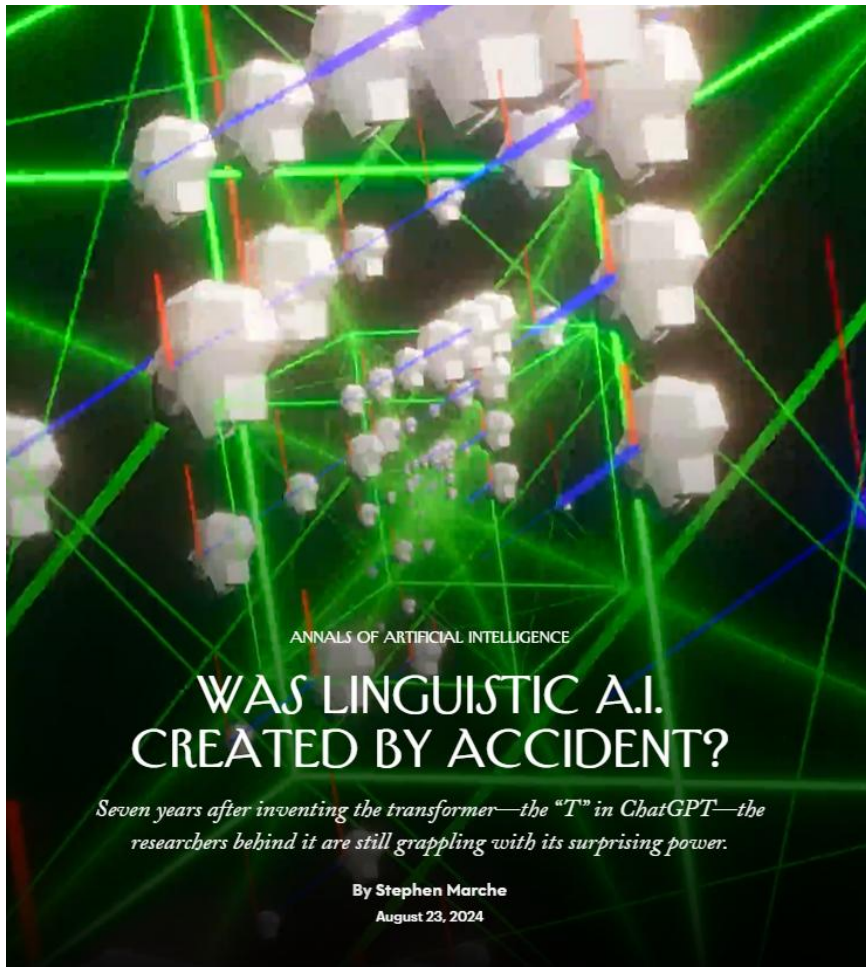
Attention was nothing more than relatively simple matrix multiplications. The chips used by Google did one thing really well: matrix multiplication. This allowed them to implement Self-attention at unprecedented speed.

Transformers

- Introducing the game-changer in the world of NLP: the Transformer model.
- Captured in a single, powerful image, this architecture redefined how we approach natural language processing and understanding. With its ingenious self-attention mechanism, the Transformer stepped beyond traditional RNNs and CNNs, enabling parallel processing and unprecedented efficiency.
- This model laid the groundwork for all the major breakthroughs that followed, from BERT to GPT, marking a new era where "Attention is All You Need" became the mantra that reshaped AI as we know it.



Chance or Plan?



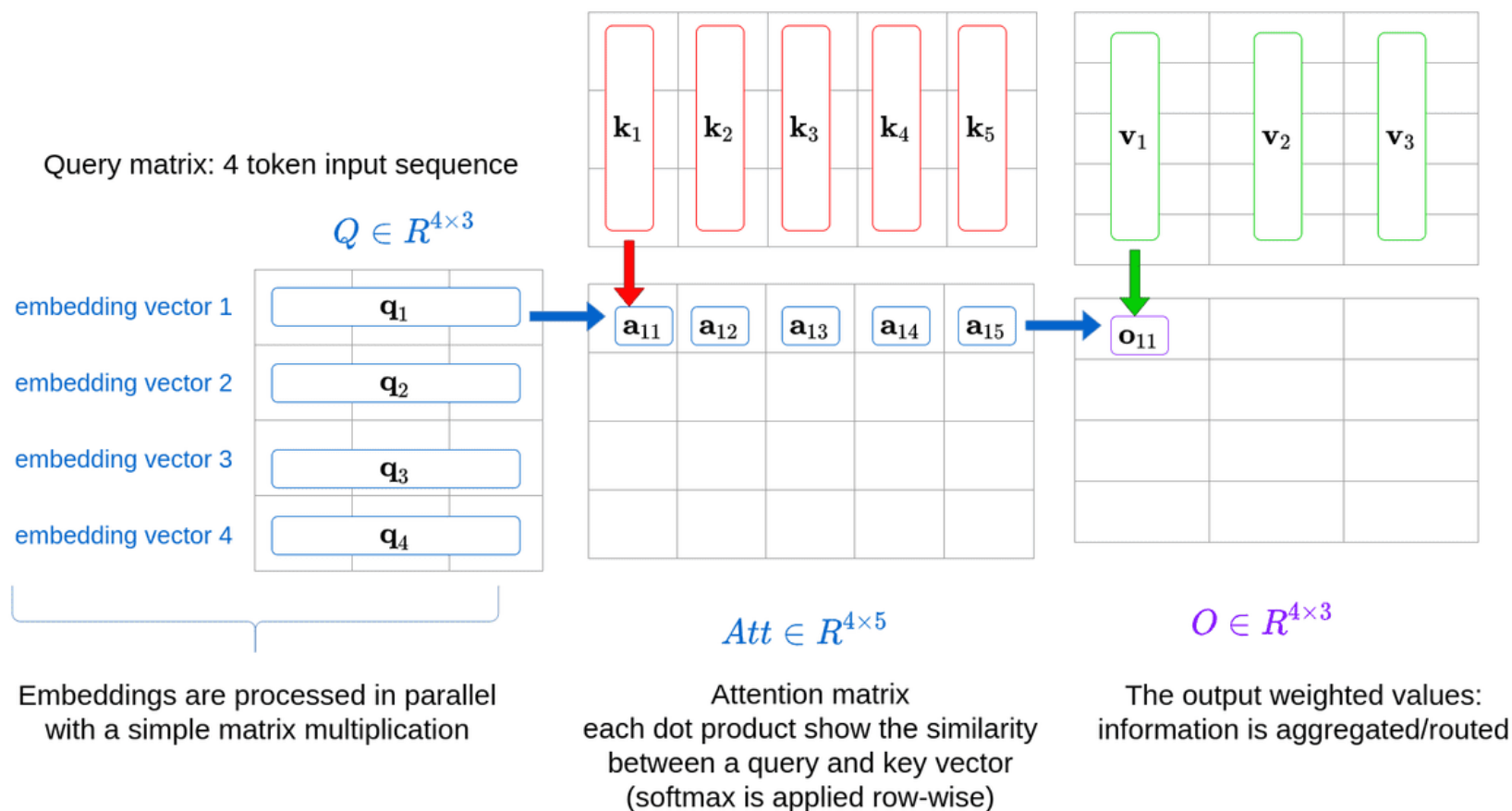
- Many great inventions were discovered by chance.
- The authors of “Attention is all you Need” did understand their computational linguistics.
- Parallel computations allowed them to scale to a level never seen before. Bigger is better? Faster is better?
- Was it chance or a well executed plan?
- The true power of the transformer only became clear a few years later around 2018-2019.
- Nobody expected a development such as ChatGPT showed in 2022 to be possible by decoding-only transformers.
- Transformers are taking over AI, not only language, but also other applications such as image recognition.

Intuition MH-SA encoding: Visually Explained

Keys, Values: 5 token sequence to associate with the input queries

$$K^T \in R^{3 \times 5}$$

$$V \in R^{5 \times 3}$$

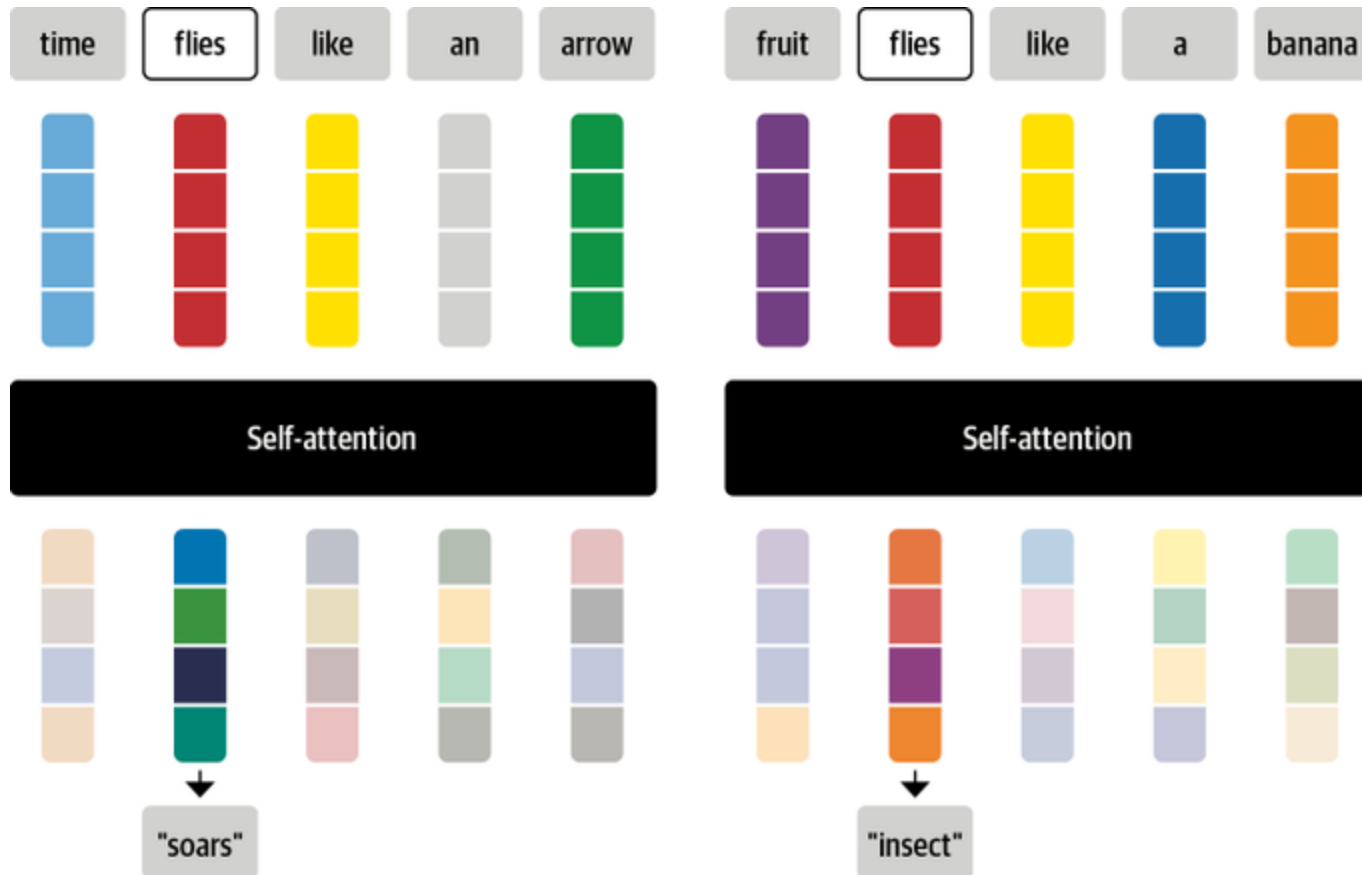


Intuition Behind Transformers Architecture

- Word embeddings:
 - Use context-sensitive encodings so tokens with similar meaning will have similar encodings. Add positional embeddings to deal with sequential properties/distances and segment or context embeddings for homonyms.
- Encoder-Decoder:
 - encoder encodes linguistic relationships in a matrix for important and less important relations implementing alignment relations.
 - This matrix is used to drive the so-called cross-attention where a basic language model (implementing next-word prediction) can be directed to select not only linguistically correct words in a particular language but also select words that are relevant given the information in the encoder (so the correct translation-, -answer or -part of abstract).
- Multi-headed Self-Attention:
 - We will discuss this a bit further, but this is used to derive the correct values in the attention matrix which we discussed above.
- Feed-Forward layers:
 - This is required to train the model using back-propagation, so only the relevant information from one layer is forwarded to the next layer (a form of 1-dimensional convolution). See it as a convolution filter that can be trained or in other words, be derived automatically from the training data. Without such filters, every layer would do the same and there would not be any need for multiple layers. By filtering, each layer also gets a particular task which will result in a more stable training process.
- Multiple layers (Nx) of self-attention:
 - by repeating the process of calculating multi-headed self-attention and feed-forward multiple times, the system is able to derive obvious visual relations in the lower layers (punctuation, morphology) and more complex relations in the middle layers (syntax, basic semantics) and complex linguistic relation in the upper levels (resolving co-references & pronouns, long distance logical relations, etc.).
- SoftMax and Linear operation at the end of the decoder:
 - Linear operation allows for the addition of a bias vector (threshold) in the training process.
 - SoftMax converts the values coming out of the decoder into normalized (probability) values between [0-1] as with other deep-learning models.



Disambiguating with Self-Attention



From Attention to Self-Attention to Multi-Headed Self-Attention

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^4 \exp(e_{ik})}$$

where $e_{ij} = \text{fc}(s_{i-1}, h_j)$

Dot product $q \cdot k_i$ attention without training (SoftMax on dot-product from 2 word embedding vectors)

Matrix multiplication after training resulting in weights for q and k

$$A(q, K, V) = \sum_i \frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}} v_i$$

Becomes: $A(Q, K, V) = \text{softmax}(QK^T)V$

$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V = Z$$

Dimensionality of word embeddings (normalization)

X = input vector,
rows = 512,
column = tokens

$$X \times W^Q = Q$$

$$X \times W^K = K$$

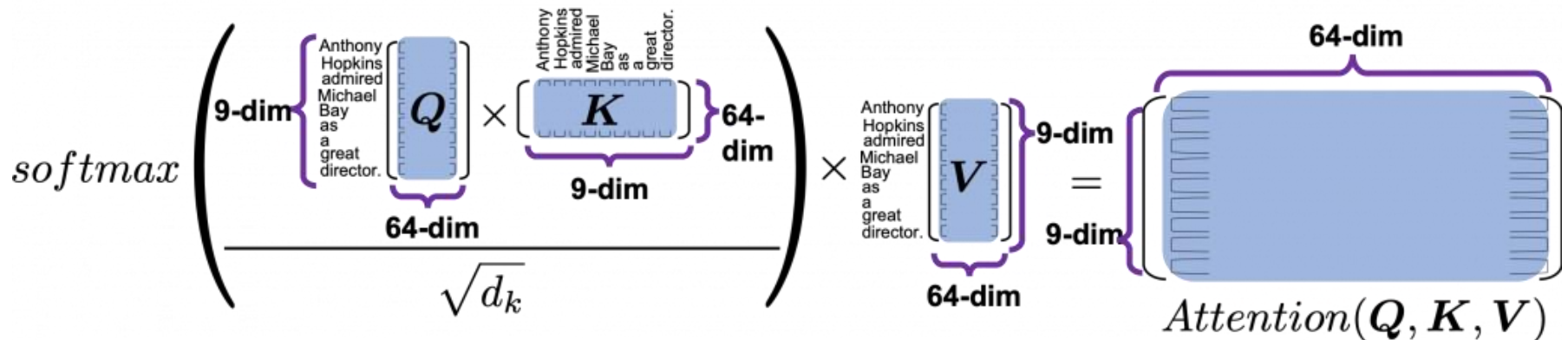
$$X \times W^V = V$$

Set by training

$$\begin{aligned} \text{head} &= \text{Attention}(QW^Q, KW^K, VW^V) \\ \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \end{aligned}$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

Calculating $\text{Attention}(Q, K, V) = \text{SoftMax}(QK^T)V$

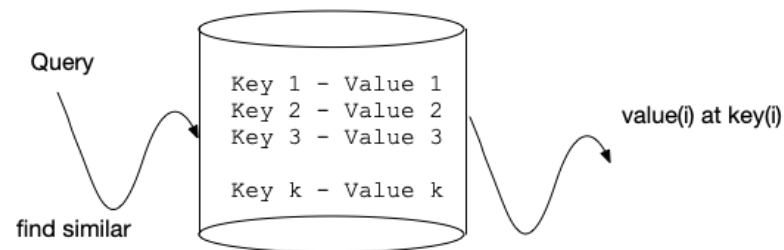


As you can see in the figure above, calculating $\text{Attention}(Q, K, V)$ is virtually just multiplying three matrices with the same size (Only K is transposed though). The resulting 9x64 matrix is the output of the head.

64 probably fits best on GPU and TPU hardware for matrix multiplications. However, I did not find any references for this hypothesis.

(Self) Attention as Queries, Keys, and Values

- When building a language model, we're interested in the similarity measure of a word with respect to every other word in the source sequence and this can be formulated as the retrieval (from a database) with a query q of a value $v(i)$ available at the key, $k(i)$.
- Dimensionality of one head is $d_k = 64$ in our example
- Q , K and V are matrices including ALL vectors for parallel calculations and to make sure we compare everything to everything.



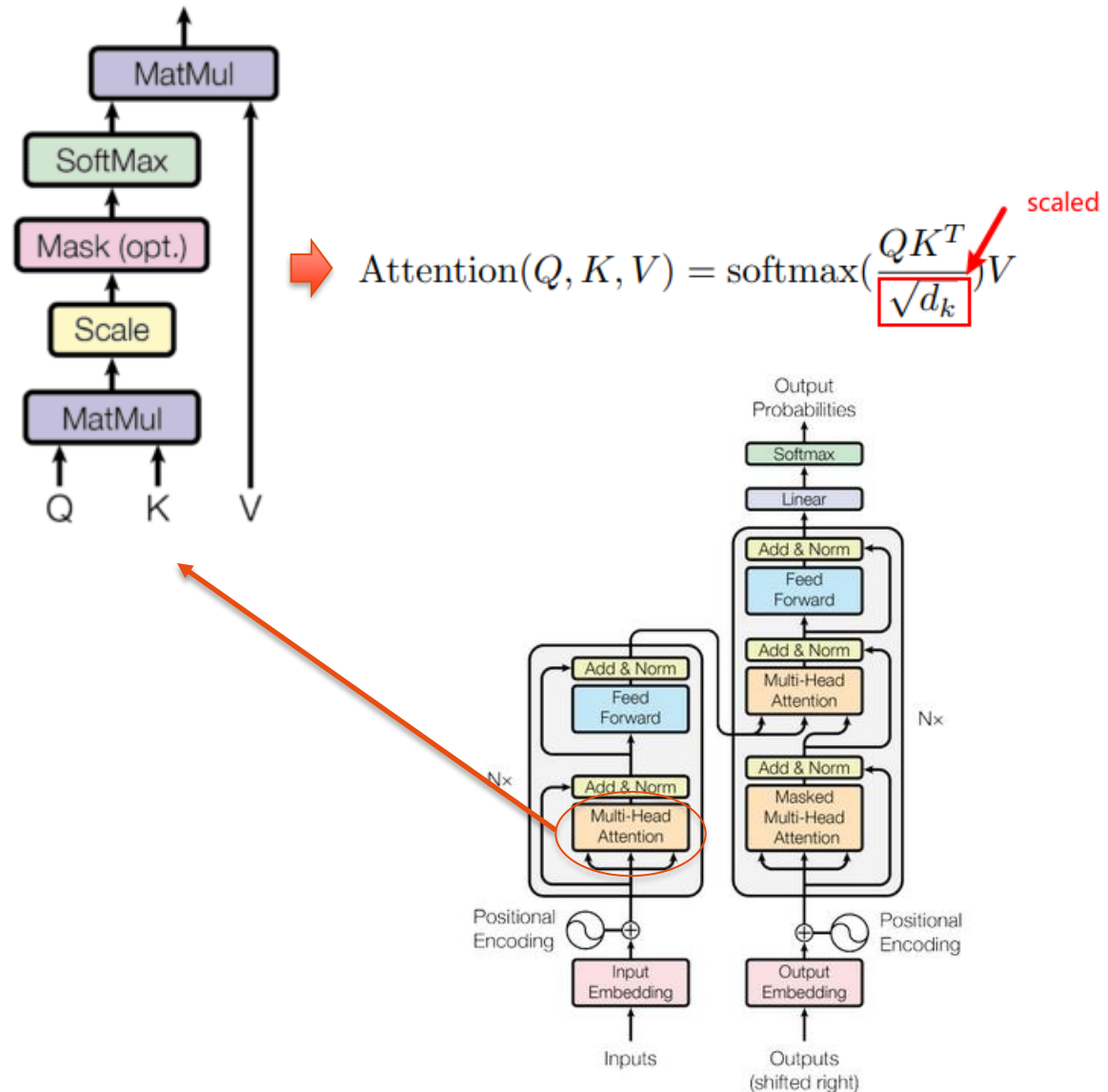
Self-Attention can be defined as a query that performs the best “match” against a set of keys that are then used to retrieve a value associated with the key.

$$\text{softmax} \left(\frac{\begin{matrix} \text{Anthony} \\ \text{Hopkins} \\ \text{admired} \\ \text{Michael} \\ \text{Bay} \\ \text{as} \\ \text{a} \\ \text{great} \\ \text{director.} \end{matrix} \begin{matrix} Q \end{matrix} \times \begin{matrix} \text{Anthony} \\ \text{Hopkins} \\ \text{admired} \\ \text{Michael} \\ \text{Bay} \\ \text{as} \\ \text{a} \\ \text{great} \\ \text{director.} \end{matrix} \begin{matrix} K \end{matrix}}{\sqrt{d_k}} \right) = \begin{matrix} \text{Attention Matrix} \end{matrix}$$

Importantly, the sum of each row is 1.

Scaled-Dot-Product Attention

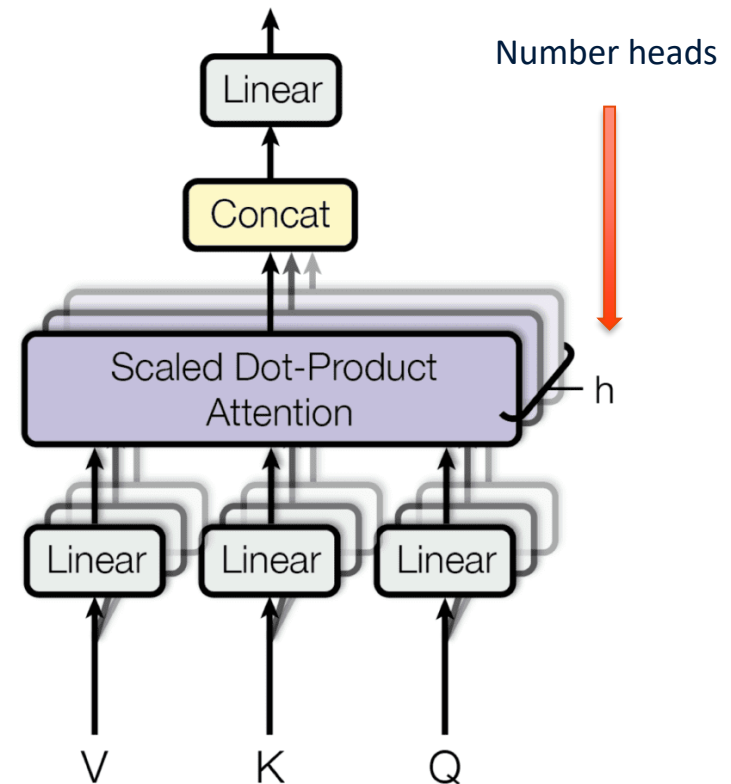
- The Transformer implements a scaled dot-product self-attention (only normalized with d_k) we defined earlier.
- Weights from matrices Q, K and V are set by training.



Multi-Headed Self-Attention

The transformer also implement a multi-headed self-attention mechanism:

- Multi-head attention mechanism linearly projects the queries, keys and values times, each time using a different learned projection. The single attention mechanism is then applied to each of these projections in parallel, to produce outputs, which in turn are concatenated and projected again to produce a final result: so, we compare everything with everything!
- The idea behind multi-head attention is to allow the attention function to extract information from different representation subspaces, which would, otherwise, not be possible with a single attention head.
- W^Q , W^K , and W^V denoting projection matrices that are used in generating different subspace representations of the query, key and value matrices. These start random, but are adjusted during training, as is W^O , the projection matrix for the multi-head output.



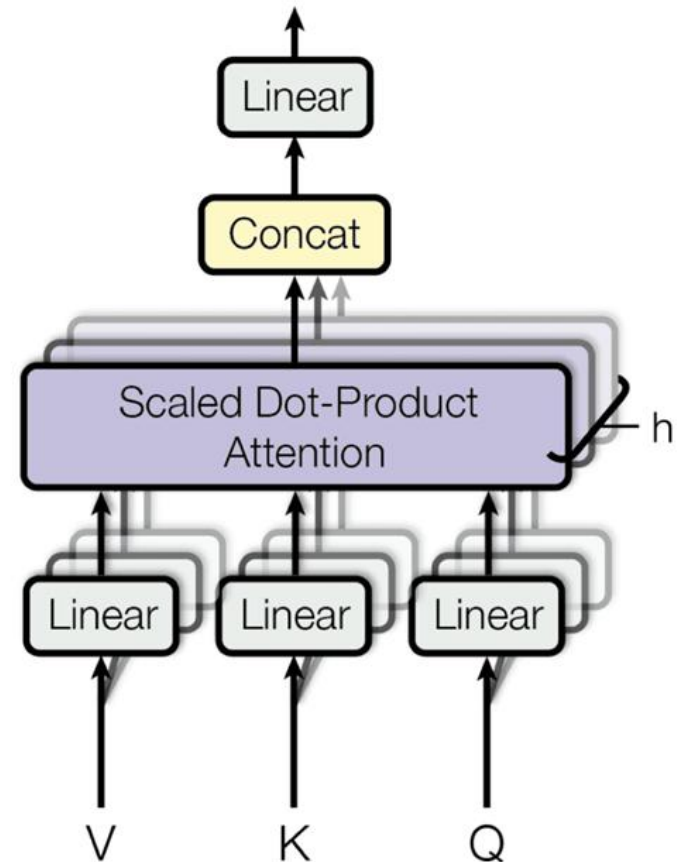
$$\text{head} = \text{Attention}(QW^Q, KW^K, VW^V)$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

Dimensionalities

- Word embeddings have dimensionality of 512 (in transformers).
- Internal embeddings BERT have dimensionality of 768.
- Typically:
 - Transformers 8 heads = h
 - BERT: 12 heads in self-attention = h
- V, K and Q have dimensionality $512/8$ or $768/12 = 64$ by “number of words in context” (max 12) or $512 / 8 = 64$ (in case of 8 heads).
- Dimension Concat = $12 \times 64 = 768$ (BERT) or 8×64 (Transformers)



MH-SA Dimensionalities

Symbol	Shape	Meaning
Q	$n \times d_k$	Query vectors (one per token)
K	$n \times d_k$	Key vectors (one per token)
V	$n \times d_v$	Value vectors (one per token)
QK^T	$n \times n$	Pairwise attention scores between tokens
$\text{softmax}(QK^T / \sqrt{d_k})$	$n \times n$	Attention weights (row-normalized)
Output: attention $\times V$	$n \times d_v$	Weighted combination of values

Step-by-step:

1. QK^T

$$(n \times d_k) \cdot (d_k \times n) \rightarrow (n \times n)$$

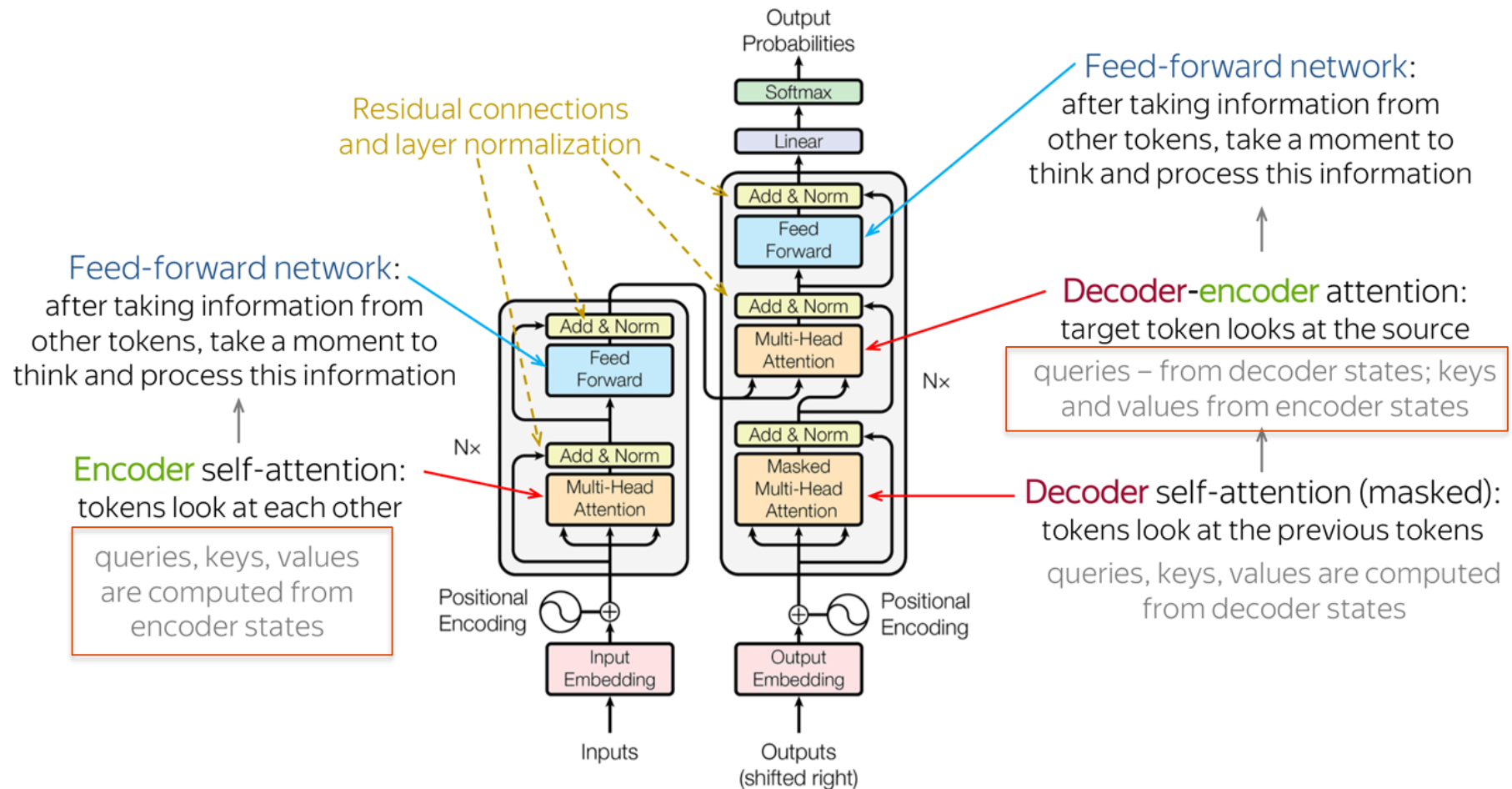
2. Softmax normalization stays $(n \times n)$.

3. Multiply with Value matrix V :

$$(n \times n) \cdot (n \times d_v) \rightarrow (n \times d_v)$$



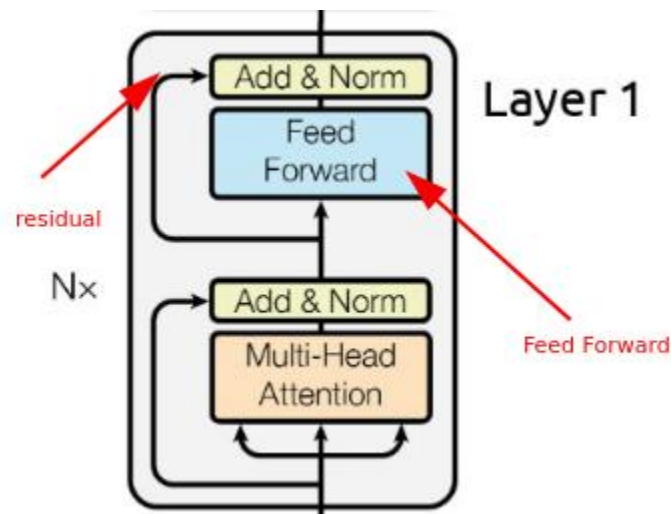
Let's Take a closer look at Transformers



What does the architecture of the Feed Forward Layers look like?

Are the trained?

- It is a position-wise transformation that consists of linear transformation, ReLU, and another linear transformation.
- The feed-forward layer has weights which are trained. The exact same matrix is applied to each respective token position.



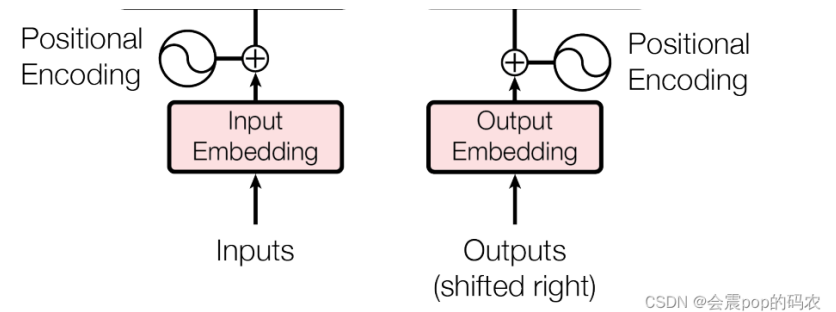
$$\text{ffLayer} = \sum_i \text{relu}(q_i k_i^T + b_i) v_i + c$$

Why Feed-Forward after Self-Attention?

- If there is no feed-forward layer, self-attention is simply performing re-averaging of value vectors.
- If there is no feed forward, all attention layers will act the same, so no added value for higher up layers.
- Adding feed forward makes each attention layer behave like a separate small model that can be trained (get parameters). Then, in the whole process of stacked self-attention, each layer gets a different “filter” by training.

Purpose of a feed forward layer is to have something to train

Input Embeddings

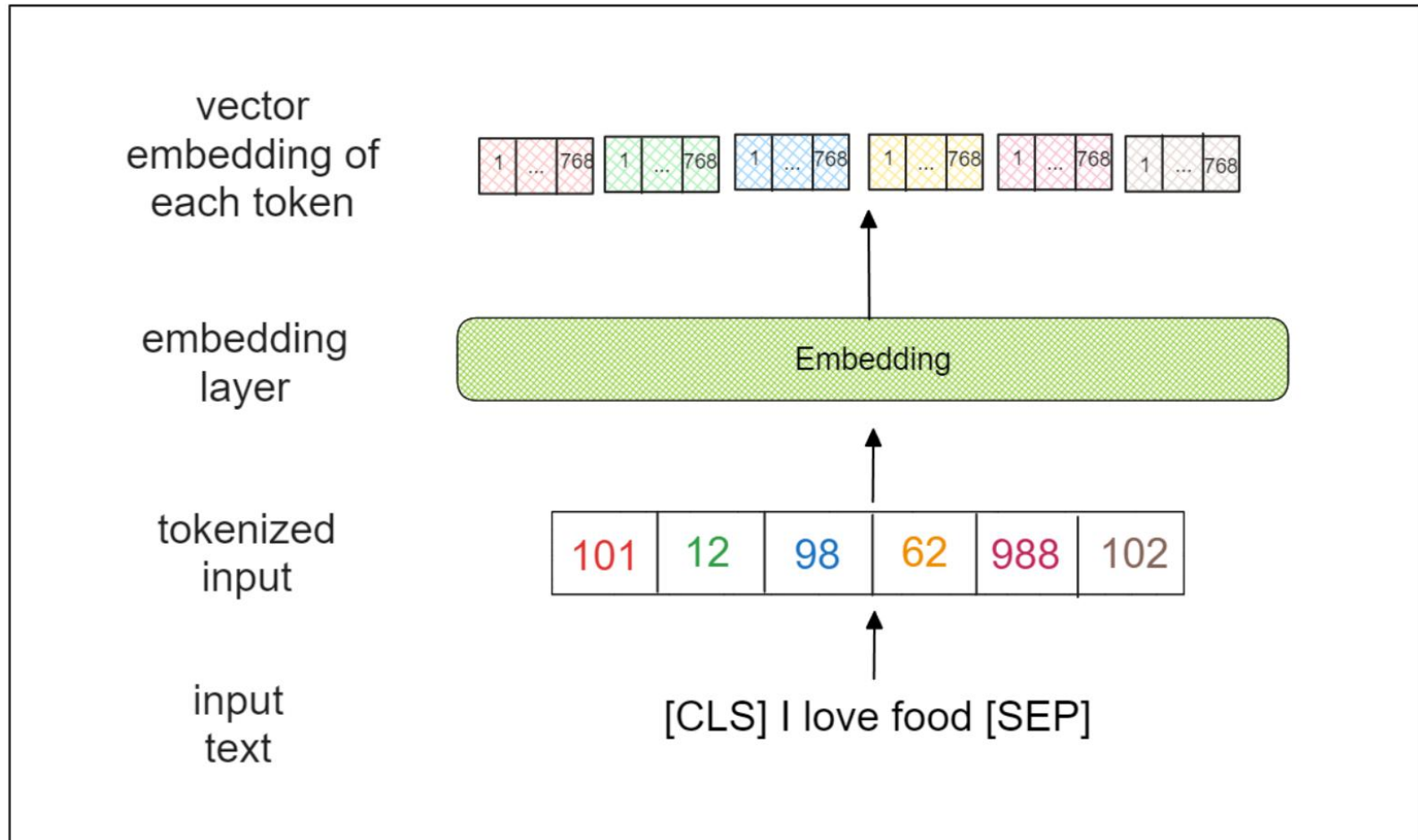


Transformer **input embeddings** are the numerical vectors the model uses as its starting point to process text. They combine two pieces of information:

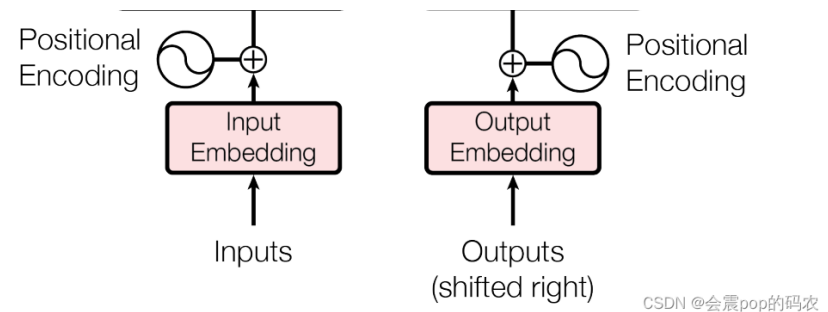
- **Token embeddings** – each input token (word, subword, or character) is mapped to a dense vector of size d_{model} via a learned lookup table.
- **Positional encodings** – since self-attention has no notion of order, a position signal is added to the token embedding (either fixed sinusoidal or learned) to tell the model where each token is in the sequence.
- $$\text{Input embedding for token } i = \text{WordEmbed}(t_i) + \text{PositionalEncoding}(i)$$

This d_{model} -dimensional vector is then fed into the first Transformer layer.

Token Embeddings



Output Embeddings



Decoder:

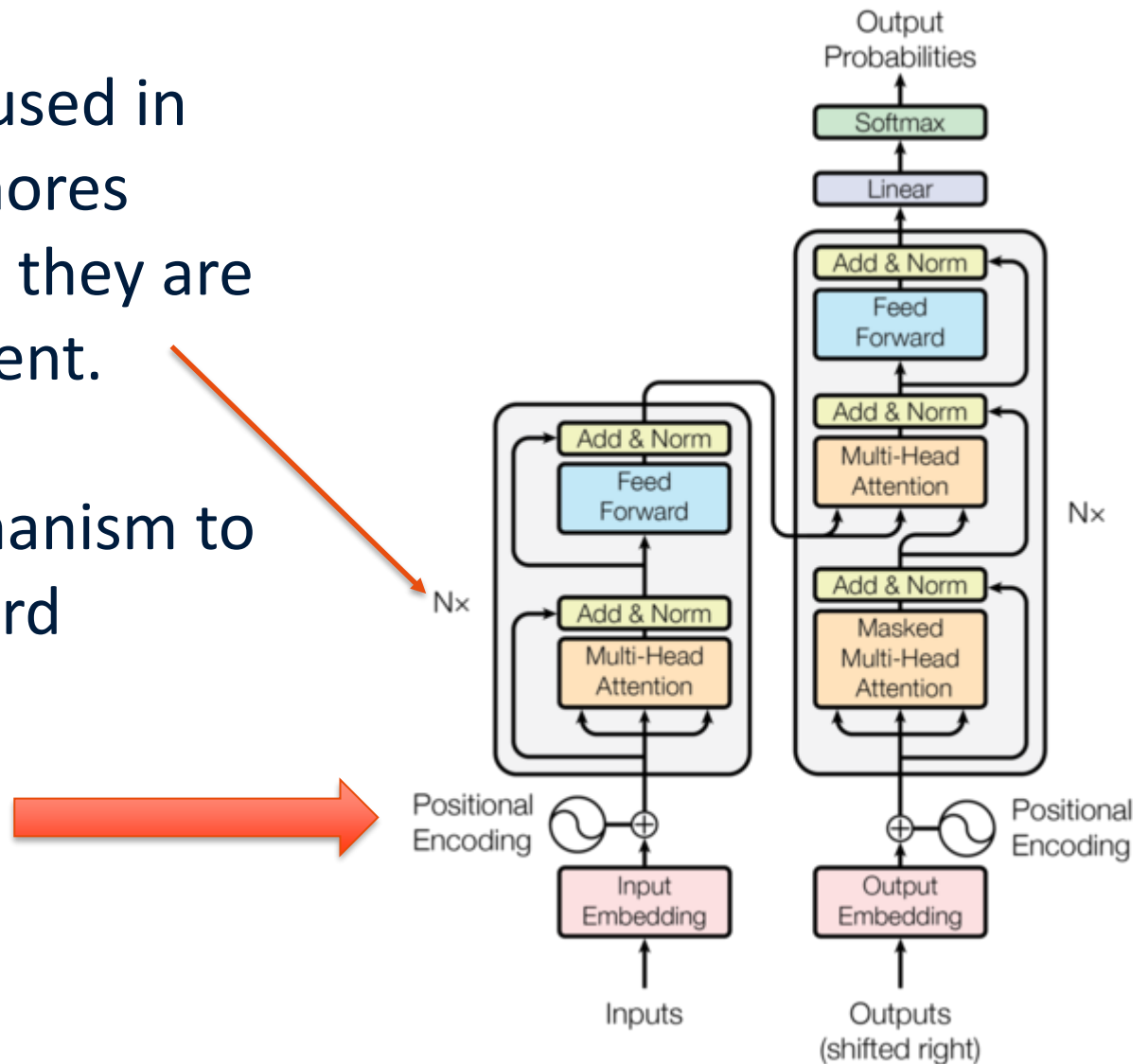
- **Output Embedding** (sometimes called *target embeddings* or *decoder input embeddings*) = token embeddings of the **already generated target tokens**, but **shifted right**.
- “Shifted right” means the decoder sees $\langle s \rangle$ (start token), then token y_1 , $y_2 \dots$, to predict $y_2, y_3, \dots$.
- This ensures the model only has access to **past tokens**, never the one it is currently predicting.

The **Output Embedding block at the bottom of the decoder** is the embedding of the **text that has already been generated**:

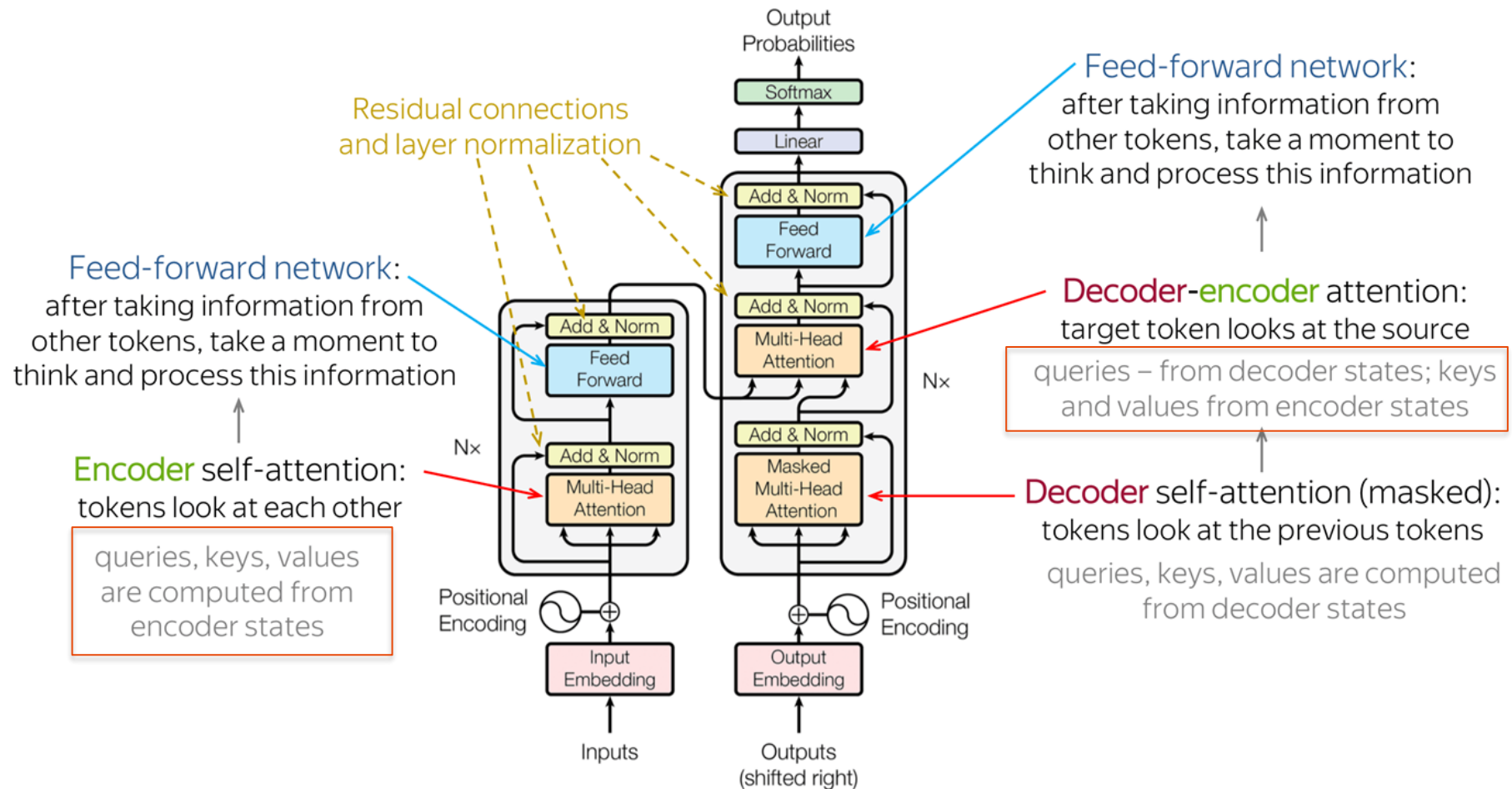
- during training: teacher-forced ground-truth tokens;
- during inference: the model’s own past outputs).

Positional Encoding in Transformers

- Representation used in Transformers ignores word position as they are no longer recurrent.
- We need a mechanism to keep track of word positions



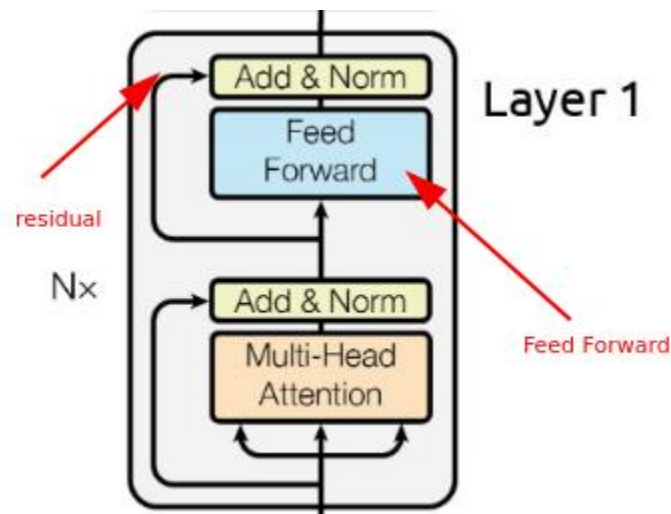
Let's Take a closer look at Transformers



What does the architecture of the Feed Forward Layers look like?

Are the trained?

- It is a position-wise transformation that consists of linear transformation, ReLU, and another linear transformation.
- The feed-forward layer has weights which are trained. The exact same matrix is applied to each respective token position.



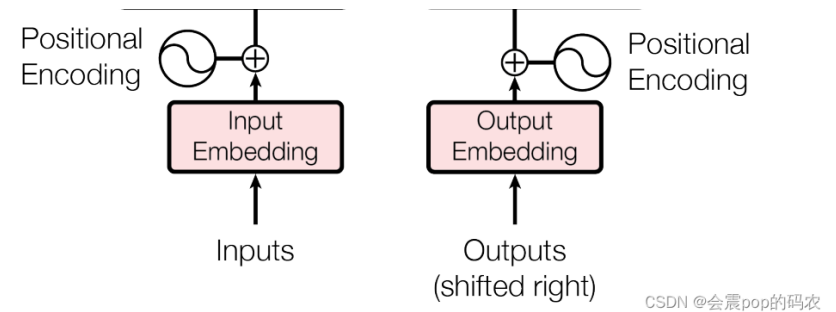
$$\text{ffLayer} = \sum_i \text{relu}(q_i k_i^T + b_i) v_i + c$$

Why Feed-Forward after Self-Attention?

- If there is no feed-forward layer, self-attention is simply performing re-averaging of value vectors.
- If there is no feed forward, all attention layers will act the same, so no added value for higher up layers.
- Adding feed forward makes each attention layer behave like a separate small model that can be trained (get parameters). Then, in the whole process of stacked self-attention, each layer gets a different “filter” by training.

Purpose of a feed forward layer is to have something to train

Input Embeddings

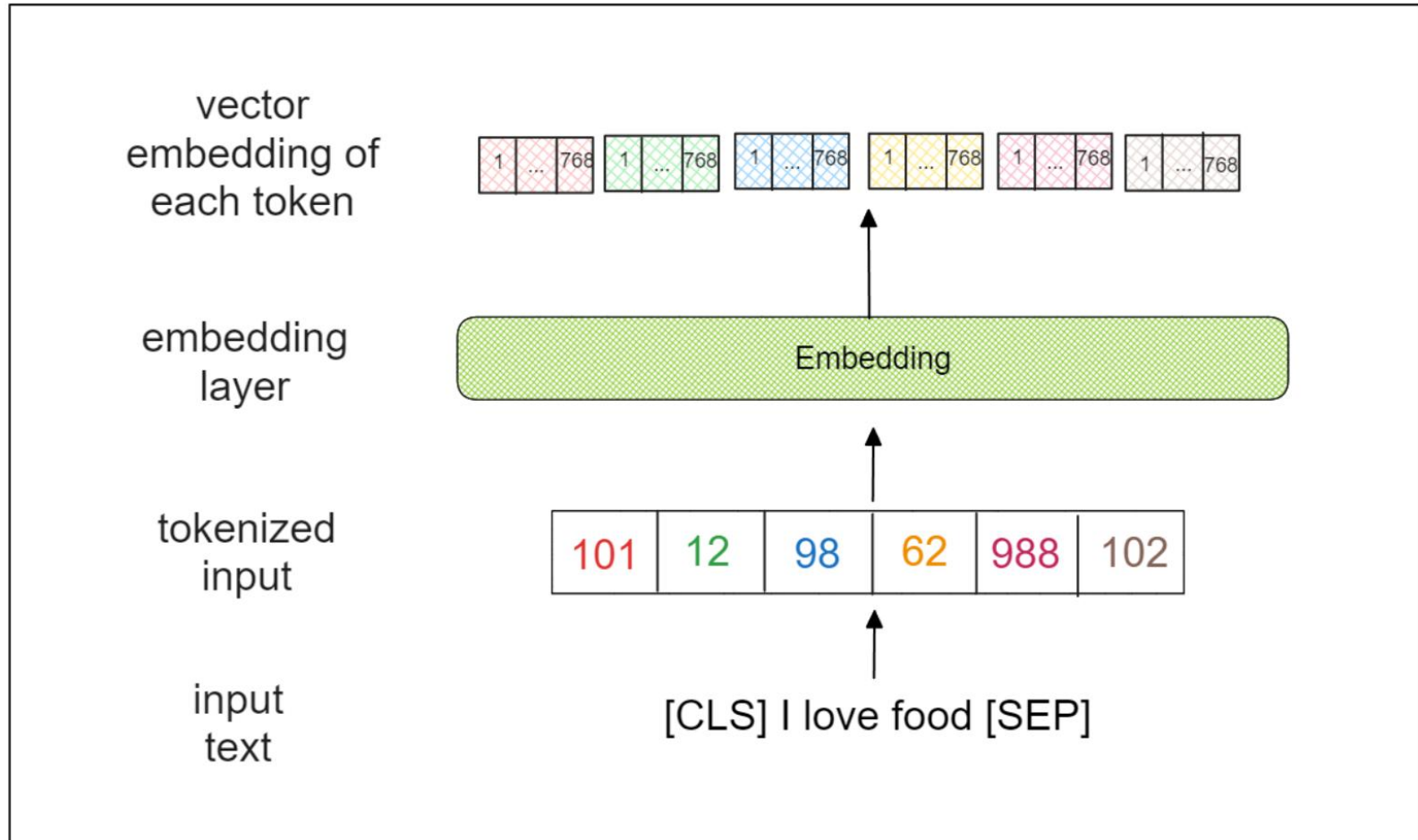


Transformer **input embeddings** are the numerical vectors the model uses as its starting point to process text. They combine two pieces of information:

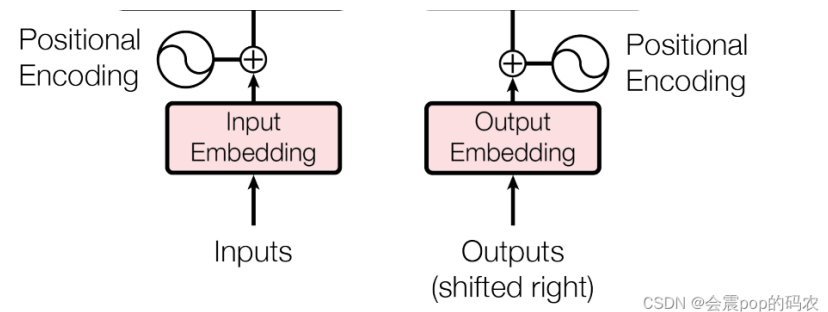
- **Token embeddings** – each input token (word, subword, or character) is mapped to a dense vector of size d_{model} via a learned lookup table.
- **Positional encodings** – since self-attention has no notion of order, a position signal is added to the token embedding (either fixed sinusoidal or learned) to tell the model where each token is in the sequence.
- $$\text{Input embedding for token } i = \text{WordEmbed}(t_i) + \text{PositionalEncoding}(i)$$

This d_{model} -dimensional vector is then fed into the first Transformer layer.

Token Embeddings



Output Embeddings



Decoder:

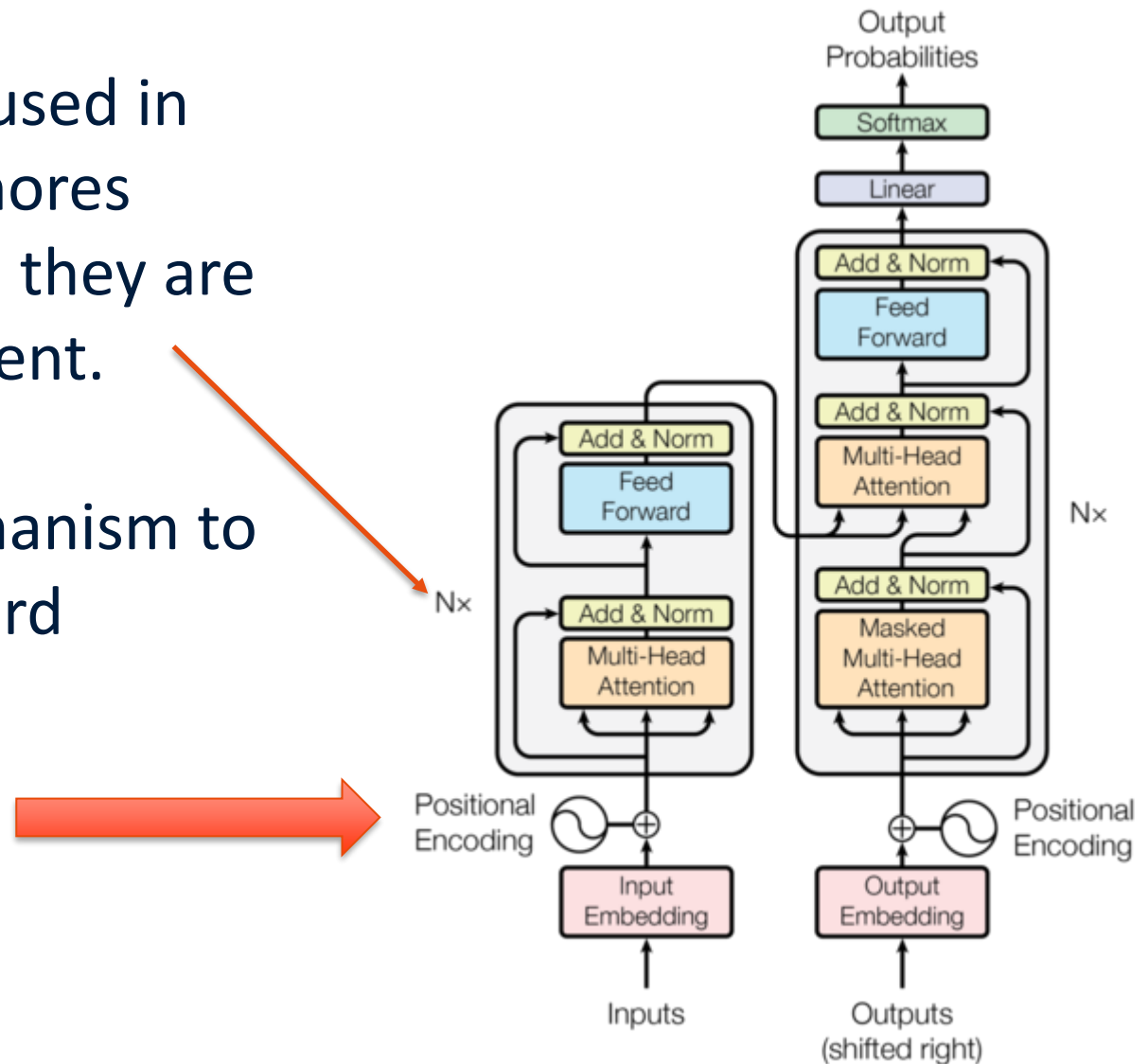
- **Output Embedding** (sometimes called *target embeddings* or *decoder input embeddings*) = token embeddings of the **already generated target tokens**, but **shifted right**.
- “Shifted right” means the decoder sees $\langle s \rangle$ (start token), then token y_1 , $y_2 \dots$, to predict $y_2, y_3, \dots$.
- This ensures the model only has access to **past tokens**, never the one it is currently predicting.

The **Output Embedding block at the bottom of the decoder** is the embedding of the **text that has already been generated**:

- during training: teacher-forced ground-truth tokens;
- during inference: the model’s own past outputs).

Positional Encoding in Transformers

- Representation used in Transformers ignores word position as they are no longer recurrent.
- We need a mechanism to keep track of word positions

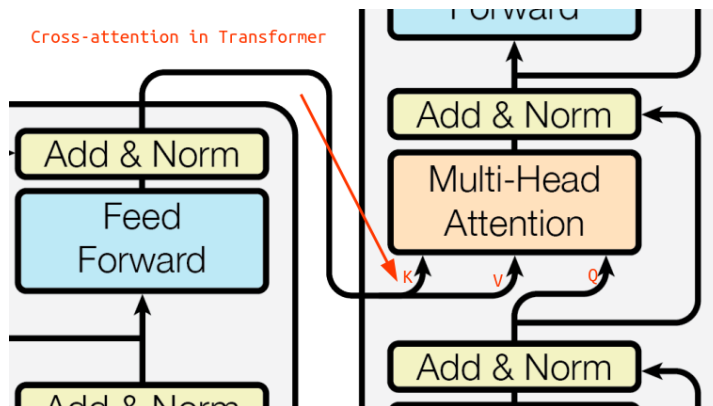


Cross Attention

- Cross-attention in transformers is a mechanism that allows the decoder to access relevant information from the encoder, enhancing tasks like translation by focusing on important details.
- It helps aligning words from the source language with their corresponding words in the target language (or text in case of abstracting) for more accurate outputs.

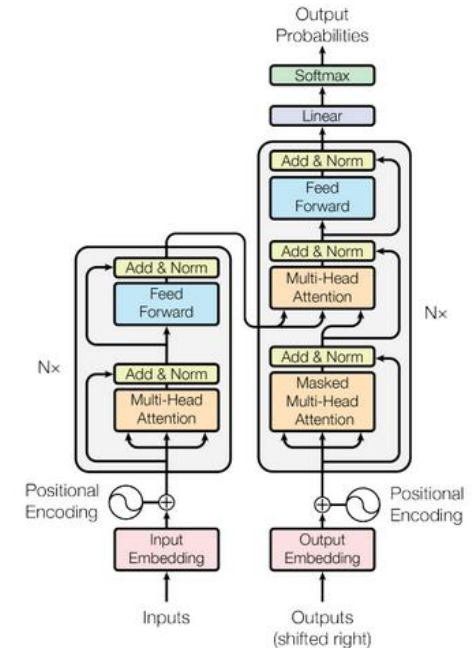
How Cross-Attention Works

1. *Encoder-Decoder Interaction:*
 - The encoder processes input data (e.g., a sentence) and generates representations.
 - The decoder uses these representations to produce output (e.g., a translated sentence).
2. *Query, Key, and Value:*
 - The decoder creates a "query" for each word it generates.
 - The encoder provides "keys" and "values" that help identify important parts of the input.
3. *Attention Calculation:*
 - The attention mechanism calculates the relevance of each key to the query, allowing the decoder to focus on the most pertinent information.



Generation Step

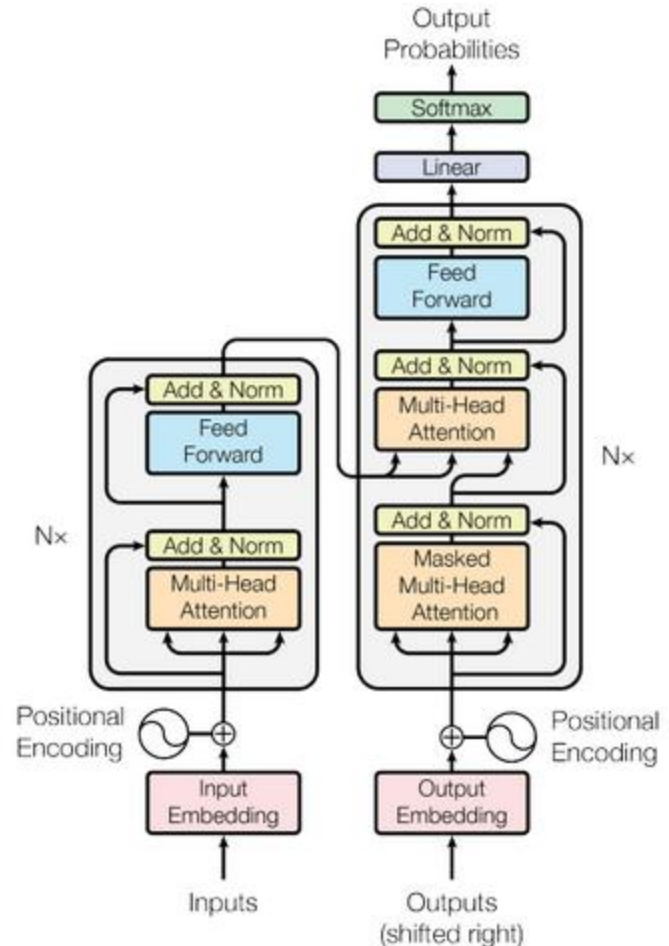
- The decoder now has a hidden state that knows both:
 - **Its own history** (via masked self-attention),
 - **The source sequence** (via cross-attention).
- A linear layer + Softmax converts this hidden state into a probability distribution over the vocabulary:
$$P(y_t, | y_{<t}X) = \text{Softmax}(W_o h_t)$$
- The model samples or chooses the most likely next token, appends it to the sequence, and repeats.



Cross-attention = the decoder “looking back” at the encoder memory to decide what to say next. At each decoding step, the query comes from what’s been generated so far; the keys/values are fixed from the encoded input. This is how the model grounds each generated token in the source sequence.

Transformer Parameters

- Input is using 768 dimensional embeddings (BERT Embeddings), one per word. 512 was the dimensionality in the original Transformer paper.
- Sentences are split into a fixed number of segments. Typically, 12-24. But in this case let's say 8.
- Number of layers, N_x is typically 6.
- Heads for multi-headed attention are the same as the number of sentence segments. Also 12-24. But we can also take 8 here.
- Then $768/12 = 64$ (dimensionality per head for BERT embeddings). Or $512 / 8 = 64$ in the original transformer setup
- Other dimension is the number of words in our window that we observe.



Computational Complexity of Transformers

Each Transformer layer has:

1. Self-attention

- Query, Key, Value projections: $O(n \cdot d^2)$
- Dot product attention: $O(n^2 \cdot d)$ because every token attends to every other token)
- Output projection: $O(n \cdot d^2)$

2. Feed-forward network (FFN)

- Two dense layers with hidden size d_{ff}
(usually $\sim 4d$): $O(n \cdot d \cdot d_{ff})$



Here:

- n = sequence length (context size)
- d = hidden dimension
- $d_{ff} \approx 4d$

Per-layer cost summary:

$$O(n^2 d) + O(nd^2) + O(ndd_{ff})$$

Since $d_{ff} \sim 4d$, the **dominant terms** are usually:

$$\text{Self-Attention: } O(n^2 d) \quad \text{FFN: } O(nd^2)$$

For **long sequences**, $n^2 d$ dominates (attention becomes the bottleneck).

For **shorter sequences but large model width**, nd^2 (FFN) can dominate.

Memory complexity:

- Attention weights: $O(n^2)$
- Activations (for backprop): $O(nd)$ per layer
- Total per layer: roughly $O(n^2 + nd)$

Computational Complexity Training Transformers

Increasing context length (n) hurts more than increasing model width (d).

Because attention scales as n^2 , FFN as d^2

This is why long context windows explode GPU memory: activations depend on n and d not on model size d

Inference (single forward pass):

$$O(n^2d + nd^2)$$

Training (forward + backward + update):

$$O(3(n^2d + nd^2))$$

Total training cost over dataset of size D and L layers:

$$6DL(n^2d + nd^2)$$

Memory:

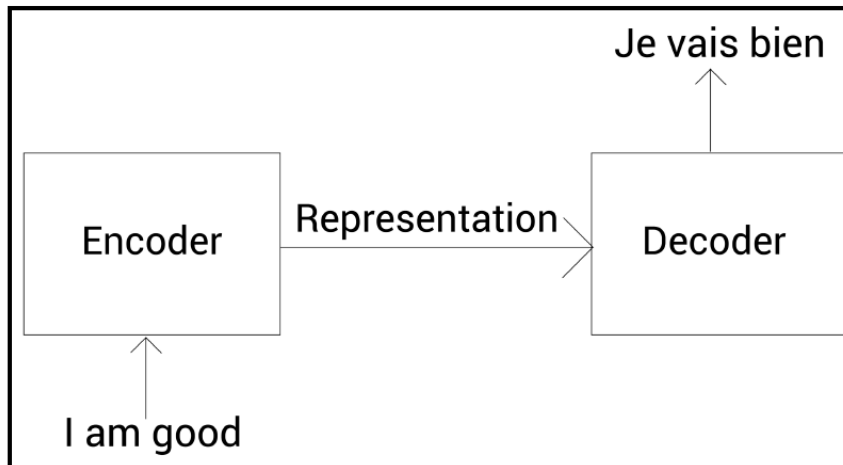
$$O(ndL + n^2)$$

Break

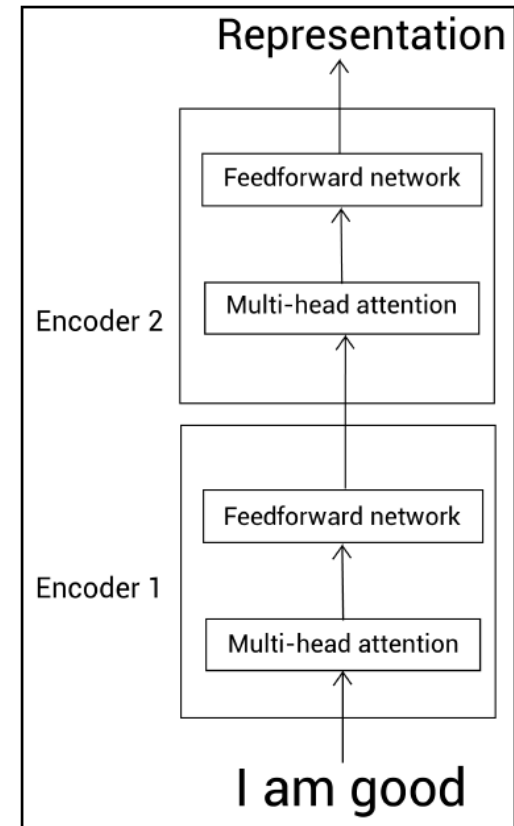
A Detailed Example of a Translation with a Transformer

I am good – Je Vais Bien

Understanding Self-Attention: detailed example



How does that work?



Multi-Headed Self Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

scaled

Let's start with self-attention, no multi-heads (yet).

$$A(q, K, V) = \sum_i \frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}} v_i$$

Becomes: $A(Q, K, V) = \text{softmax}(QK^T)V$

$$[|Q| \times d_k] \times [d_k \times |K|] \times [|K| \times d_v]$$

softmax
row-wise



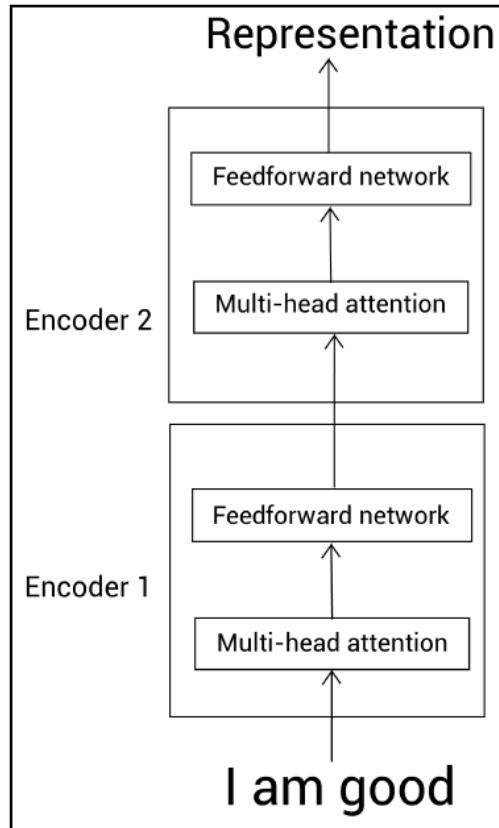
$$= [|Q| \times d_v]$$



$$\begin{aligned} \text{head} &= \text{Attention}(QW^Q, KW^K, VW^V) \\ \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \end{aligned}$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

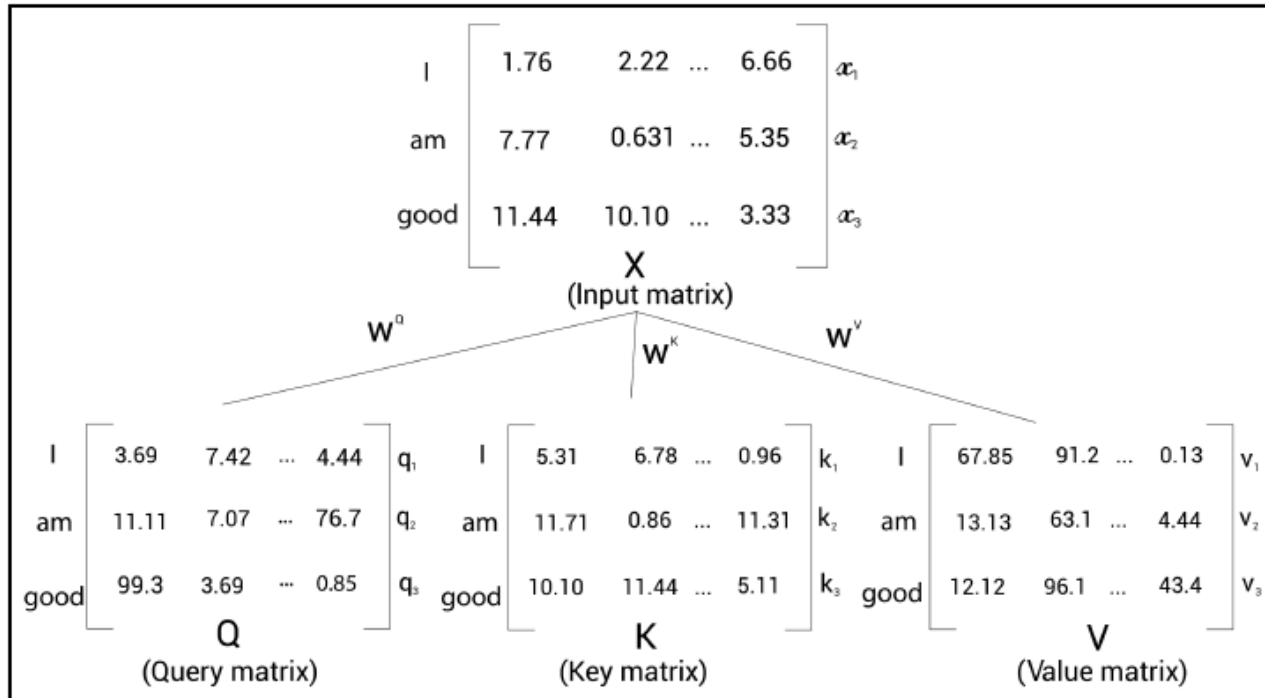
Start with the Encoder: The input embeddings



I	1.76	2.22	...	6.66	x_1
am	7.77	0.631	...	5.35	x_2
good	11.44	10.10	...	3.33	x_3
					3x512

X
input matrix
(embedding matrix)

Creating Q, K and V



Q, K and V are (3 x 64) matrices, created from matrix X (3 x 512) by multiplying X with weight matrices (initialized randomly, adjusted by training) W^Q , W^K and W^V each (3 x 64). For ease of this explanation, we presume that the values above for Q, K and V are the outcome of X with W^Q , W^K and W^V .

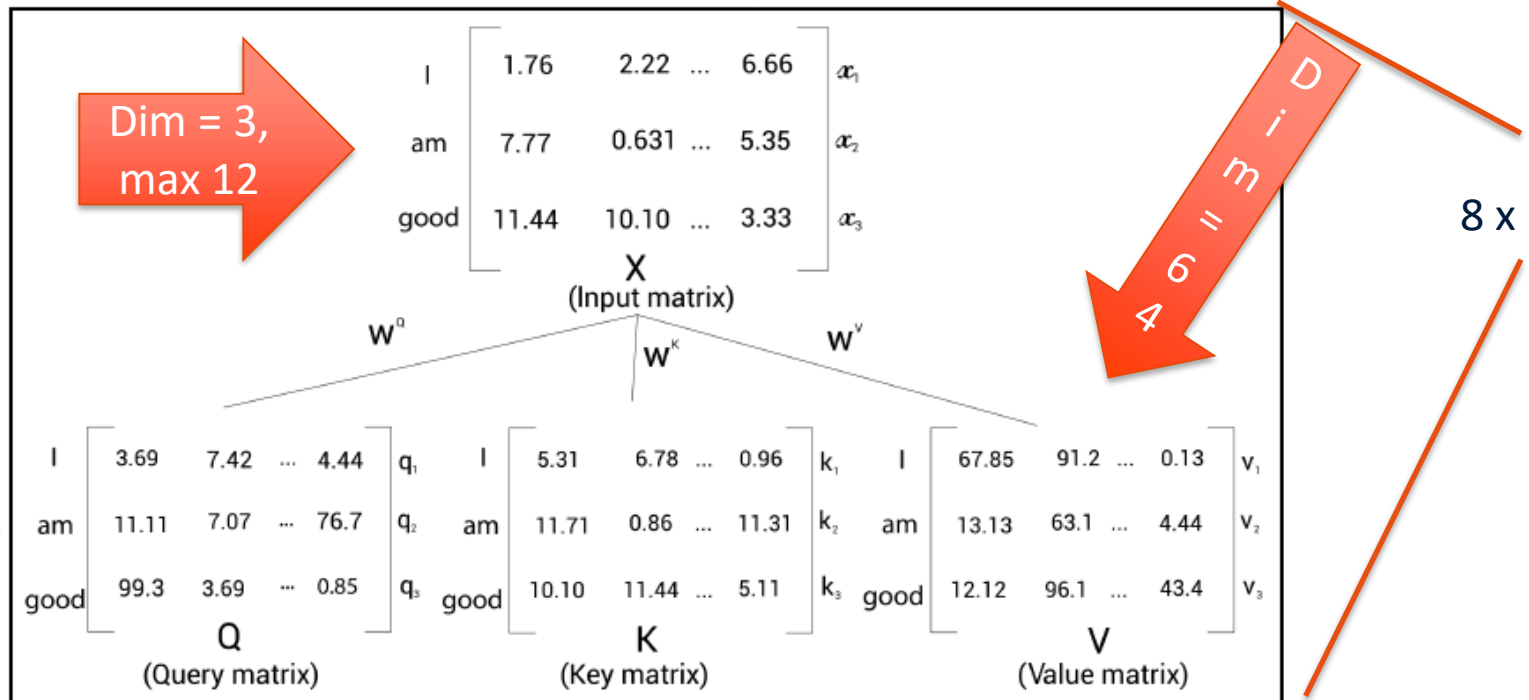
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

scaled

Creating Q, K and V

Dim
=
512

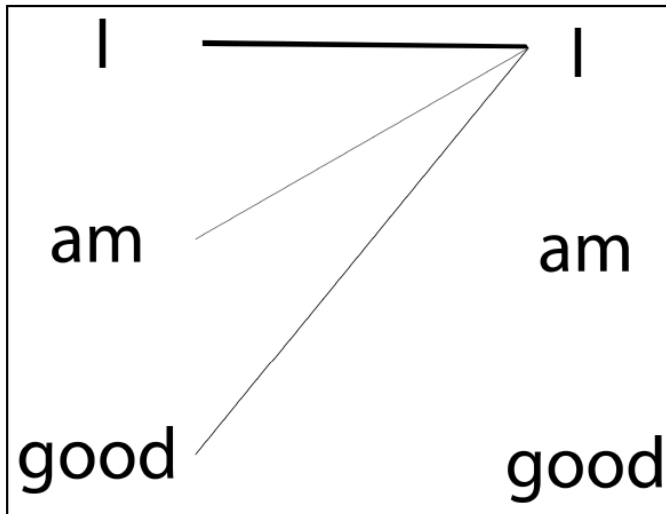
8 HEADS



Q, K and V are (3 x 64) matrices, created from matrix X (3 x 512) by multiplying X with weight matrices (initialized randomly, adjusted by training) W^Q , W^K and W^V each (3 x 64).
We have 8 of such “heads”

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

scaled



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

scaled

	I	am	good
I	3.69	7.42 ...	4.44
am	11.11	7.07 ...	76.7
good	99.3	3.69 ...	0.85

Q

	I	am	good
q_1	5.31	11.71	10.10
q_2	6.78	0.86	11.44
q_3	0.96	11.31	5.11
	k_1	k_2	k_3

K^T

$$\begin{array}{c} \text{I} \\ \text{am} \\ \text{good} \end{array} \begin{bmatrix} 3.69 & 7.42 & \dots & 4.44 \\ 11.11 & 7.07 & \dots & 76.7 \\ 99.3 & 3.69 & \dots & 0.85 \end{bmatrix} \begin{matrix} q_1 \\ q_2 \\ q_3 \end{matrix} \cdot \begin{matrix} \text{I} & \text{am} & \text{good} \\ \begin{bmatrix} 5.31 & 11.71 & 10.10 \\ 6.78 & 0.86 & 11.44 \\ \vdots & \vdots & \vdots \\ 0.96 & 11.31 & 5.11 \end{bmatrix} \\ k_1 & k_2 & k_3 \end{matrix} = \begin{matrix} \text{I} & \text{am} & \text{good} \\ \begin{bmatrix} q_1.k_1 & q_1.k_2 & q_1.k_3 \\ q_2.k_1 & q_2.k_2 & q_2.k_3 \\ q_3.k_1 & q_3.k_2 & q_3.k_3 \end{bmatrix} \end{matrix}$$

Q
 K^T

$$QK^T = \begin{matrix} & \text{I} & \text{am} & \text{good} \\ \begin{matrix} \text{I} \\ \text{am} \\ \text{good} \end{matrix} & \begin{bmatrix} 110 & 90 & 80 \\ 70 & 99 & 70 \\ 90 & 70 & 100 \end{bmatrix} \end{matrix}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

↗ scaled

Rounded down numbers

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

scaled

$$\frac{QK^T}{\sqrt{d_k}} = \frac{QK^T}{8} =$$

	I	am	good
I	13.75	11.25	10
am	8.75	12.375	8.75
good	11.25	8.75	12.5

$$\text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) =$$

	I	am	good
I	0.90	0.07	0.03
am	0.025	0.95	0.025
good	0.21	0.03	0.76

Sum = 1

Values V set by
training



				I	am	good		
Z =	I	0.90	0.07	0.03	I	67.85	91.2 ... 0.13	V_1
	am	0.025	0.95	0.025	am	13.13	63.1 ... 4.44	V_2
	good	0.21	0.03	0.76	good	12.12	96.1 ... 43.4	V_3
	$\text{softmax}\left(\frac{QK^T}{\sqrt{d_K}}\right)$				V			

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

scaled

← Output = a matrix Z

Self-Attention Values

Z is the output of the self-attention operation for the entire sequence.

Since you have 3 tokens in the sentence "*I am good*", self-attention outputs 3 vectors:

$$Z = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix}$$

Each z_i corresponds to **one token**, and contains that token's *new contextual representation*.

What z_1, z_2, z_3 are

Each z_i is a **vector**, not a single number.

$$z_1 = \sum_{j=1}^3 (\text{attention weight between "I" and token } j) \cdot v_j$$

Example from your slide:

- For z_1 :
"I" mostly attends to "I": weight = **0.90**
- For z_2 :
"am" mostly attends to "am": weight = **0.95**
- For z_3 :
"good" mostly attends to "good": weight = **0.76**

So:

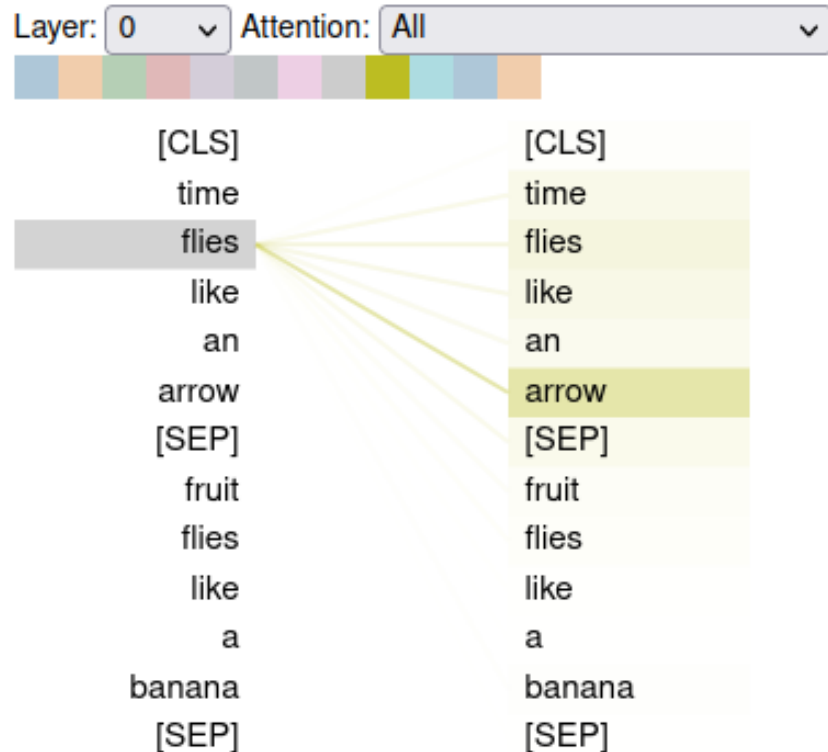
- z_1 = self-attention vector for "I"
- z_2 = self-attention vector for "am"
- z_3 = self-attention vector for "good"

Each z_i is computed as:

$$z_i = \text{softmax}(q_i K^T) V$$

Multi-Headed Attention

- Instead of calculating one matrix Z , we can calculate multiple matrices Z .
- Multi-headed self-attention will allow us to disambiguate the meaning of “ambiguous” words better because we have multiple sources to use for the disambiguation.
- We will take a closer look at this in the tutorial.



$$\text{head} = \text{Attention}(QW^Q, KW^K, VW^V)$$
$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

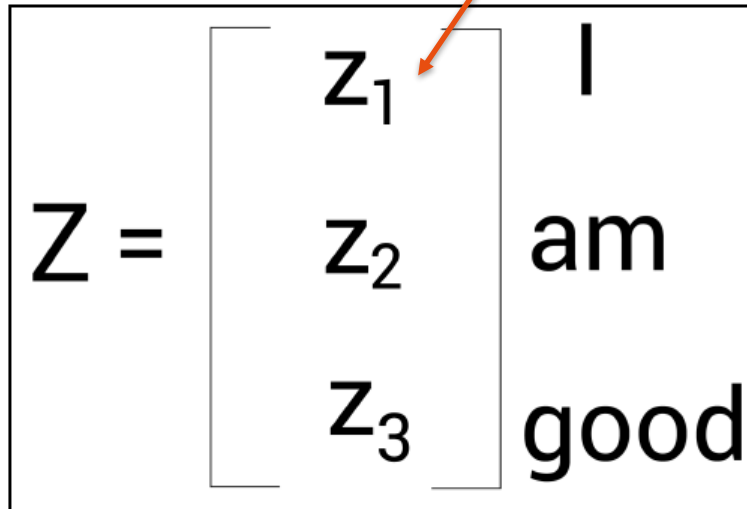
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

Self-attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

scaled

Z =
Matrix



Suppose we have this sentence “I am good”

THEN:

$$z_1 = 0.90 \begin{bmatrix} 67.85 & 91.2 & \dots \end{bmatrix} + 0.07 \begin{bmatrix} 13.13 & 63.1 & \dots \end{bmatrix} + 0.03 \begin{bmatrix} 12.12 & 96.1 & \dots \end{bmatrix}$$

$v_1(\text{I})$ $v_2(\text{am})$ $v_3(\text{good})$

Self-attention “I”

$$z_2 = 0.025 \begin{bmatrix} 67.85 & 91.2 & \dots \end{bmatrix} + 0.95 \begin{bmatrix} 13.13 & 63.1 & \dots \end{bmatrix} + 0.025 \begin{bmatrix} 12.12 & 96.1 & \dots \end{bmatrix}$$

$v_1(\text{I})$ $v_2(\text{am})$ $v_3(\text{good})$

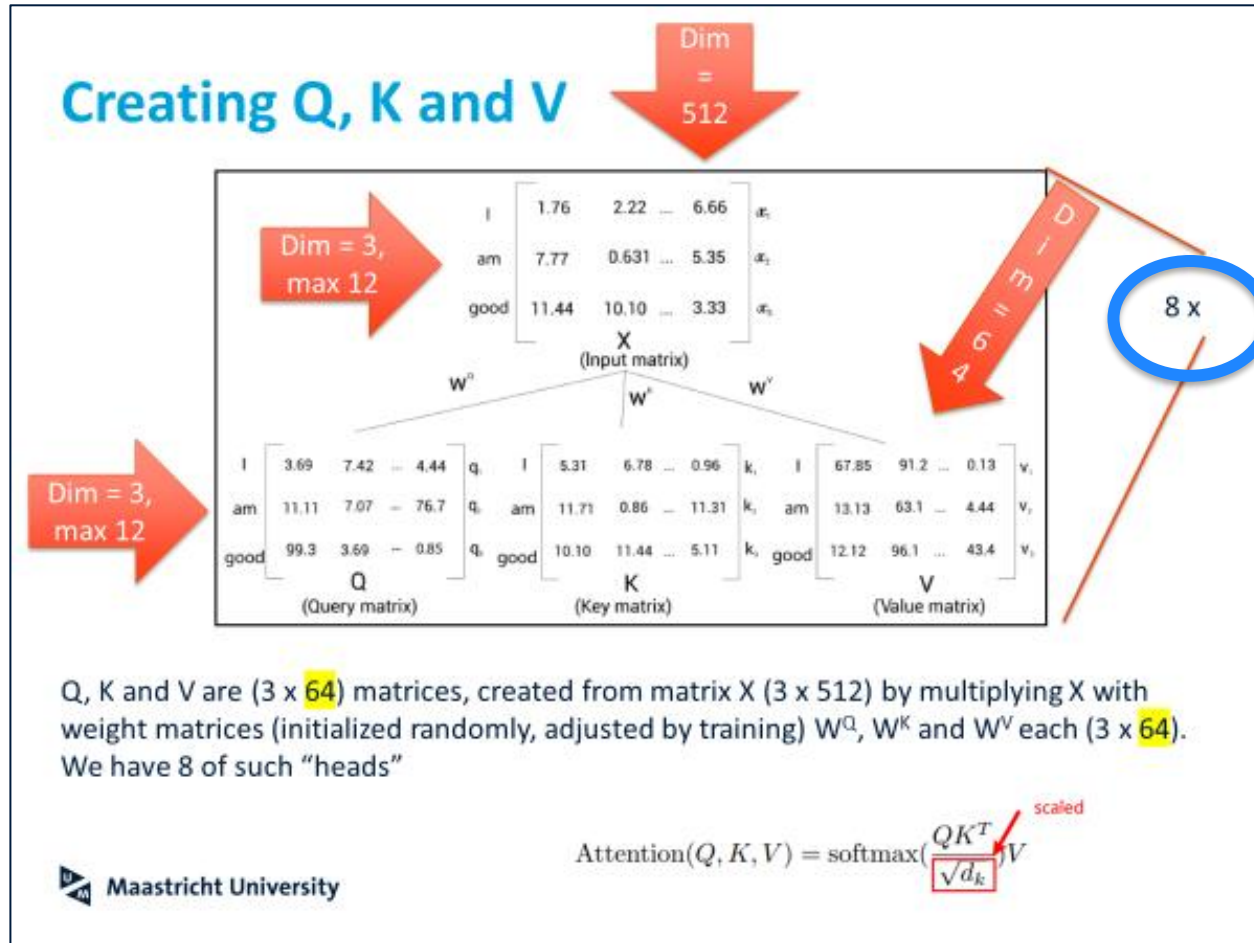
Self-attention “am”

$$z_3 = 0.21 \begin{bmatrix} 67.85 & 91.2 & \dots \end{bmatrix} + 0.03 \begin{bmatrix} 13.13 & 63.1 & \dots \end{bmatrix} + 0.76 \begin{bmatrix} 12.12 & 96.1 & \dots \end{bmatrix}$$

$v_1(\text{I})$ $v_2(\text{am})$ $v_3(\text{good})$

Self-attention “good”

So, we do this for each head....



Now let's insert the Positional Encoding

$$\begin{array}{rcl}
 & | & \begin{bmatrix} 1.769 & 2.22 & 3.4 & 5.8 \\ 7.3 & 9.9 & 8.5 & 7.1 \\ 9.1 & 7.1 & 0.85 & 10.1 \end{bmatrix} \begin{matrix} x_1 \\ x_2 \\ x_3 \end{matrix} \\
 X = & \begin{matrix} \text{am} \\ \text{good} \end{matrix} &
 \end{array}$$

Input matrix
(embedding matrix)

$$\begin{array}{rcl}
 X = & \begin{bmatrix} 1.769 & 2.22 & 3.4 & 5.8 \\ 7.3 & 9.9 & 8.5 & 7.1 \\ 9.1 & 7.1 & 0.85 & 10.1 \end{bmatrix} & + \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0.841 & 0.54 & 0.01 & 0.99 \\ 0.909 & -0.416 & 0.02 & 0.99 \end{bmatrix} \\
 & X & P \\
 = & \begin{bmatrix} 1.769 & 3.22 & 3.4 & 6.8 \\ 8.14 & 10.44 & 8.51 & 8.09 \\ 10.0 & 6.68 & 0.87 & 11.09 \end{bmatrix} &
 \end{array}$$

Positional Information

Remember: Positional Encoding

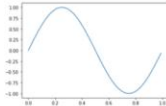
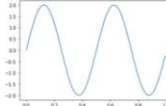
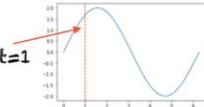
$$P = \begin{matrix} & \begin{matrix} | & \sin(\text{pos}) & \cos(\text{pos}) & \sin\left(\frac{\text{pos}}{100}\right) & \cos\left(\frac{\text{pos}}{100}\right) \end{matrix} \\ \begin{matrix} \text{I} \\ \text{am} \\ \text{a} \\ \text{Robot} \end{matrix} & \begin{matrix} \rightarrow 0 \\ \rightarrow 1 \\ \rightarrow 2 \\ \rightarrow 3 \end{matrix} & \rightarrow \end{matrix}$$

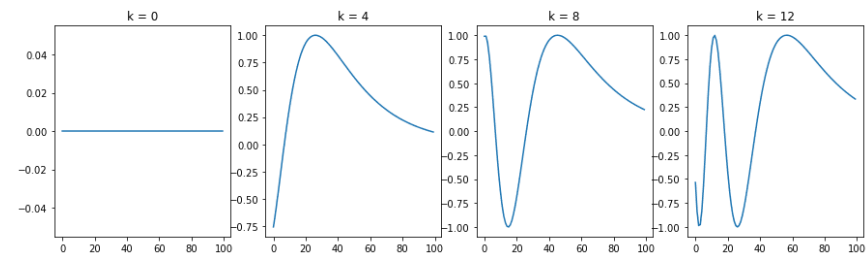
$$P = \begin{matrix} & \begin{matrix} | & \sin(0) & \cos(0) & \sin(0/100) & \cos(0/100) \end{matrix} \\ \begin{matrix} \text{I} \\ \text{am} \\ \text{a} \\ \text{Robot} \end{matrix} & \begin{matrix} \rightarrow 0 \\ \rightarrow 1 \\ \rightarrow 2 \\ \rightarrow 3 \end{matrix} & \rightarrow \end{matrix}$$

$$P = \begin{matrix} & \begin{matrix} | & 0 & 1 & 0 & 1 \end{matrix} \\ \begin{matrix} \text{I} \\ \text{am} \\ \text{a} \\ \text{Robot} \end{matrix} & \begin{matrix} \rightarrow 0 \\ \rightarrow 1 \\ \rightarrow 2 \\ \rightarrow 3 \end{matrix} & \rightarrow \end{matrix}$$

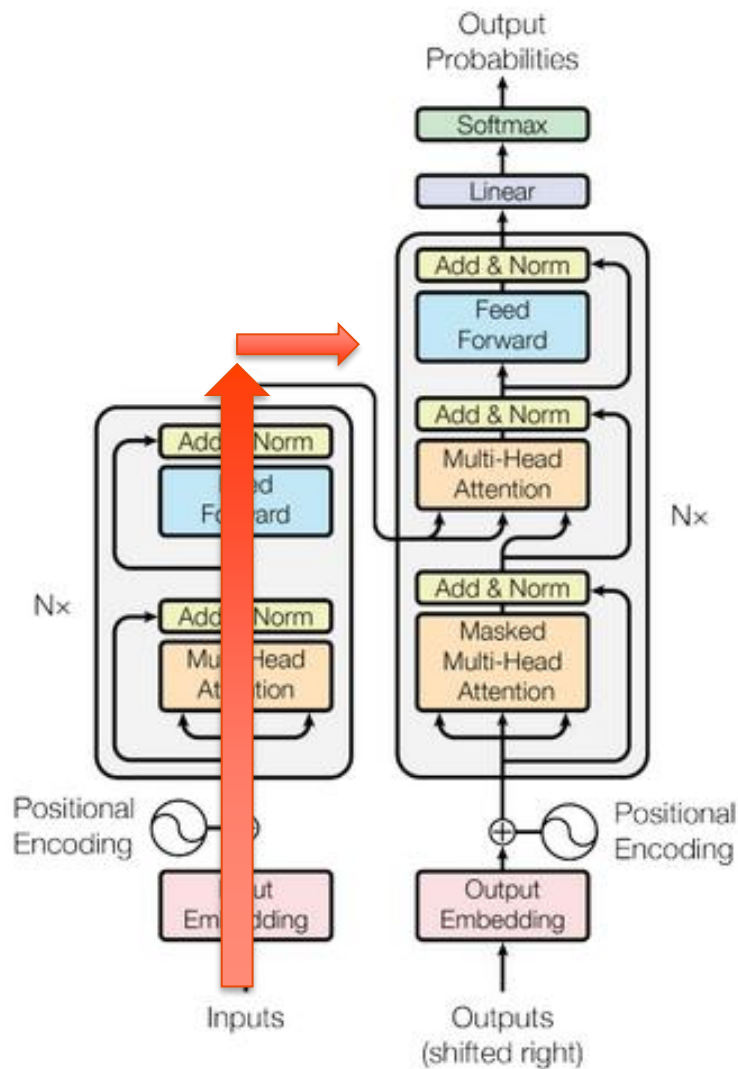
Sequence	Index of token	Positional Encoding Matrix			
I	0	P ₀₀	P ₀₁	...	P _{0d}
am	1	P ₁₀	P ₁₁	...	P _{1d}
a	2	P ₂₀	P ₂₁	...	P _{2d}
Robot	3	P ₃₀	P ₃₁	...	P _{3d}

Positional Encoding Matrix for the sequence 'I am a robot'

Equation	Graph	Frequency	Wavelength
$\sin(2\pi t)$		1	1
$\sin(2 * 2\pi t)$		2	1/2
$\sin(t)$		1/2π	2π
$\sin(ct)$	Depends on c	c/2π	2π/c

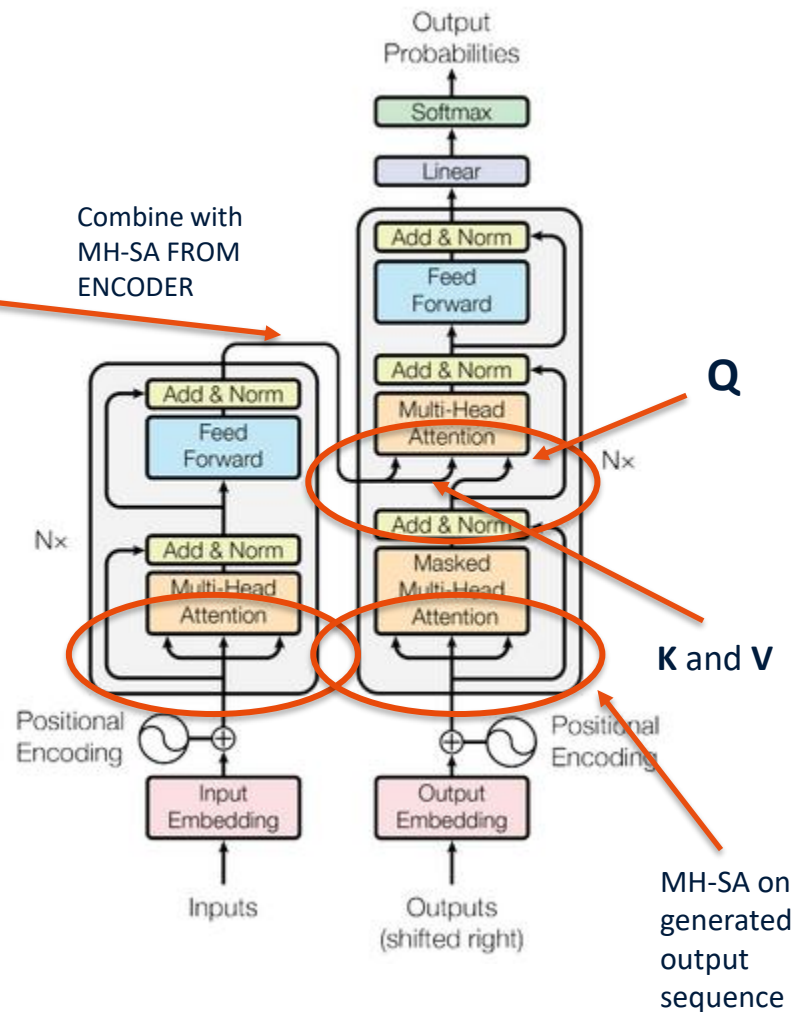


So far for the encoding, now up to the decoding



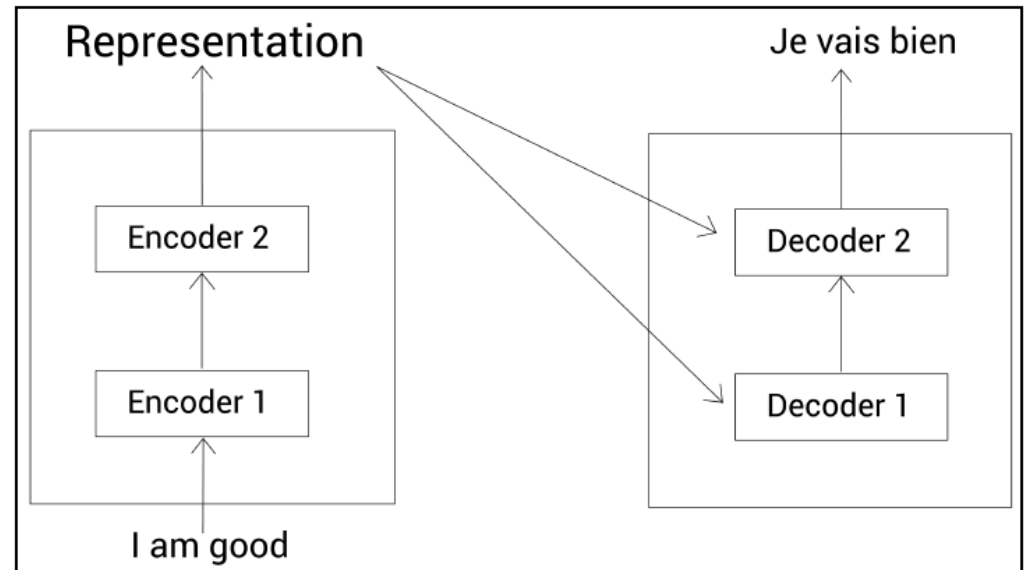
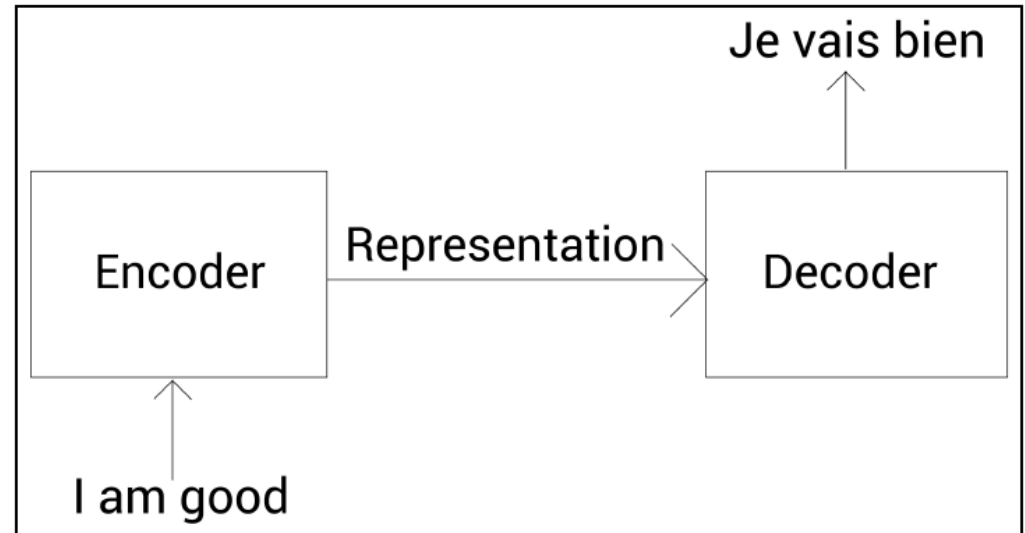
Remember decoding self-attention (aka cross attention)...

- In encoding-decoding, the encoding creates a multi-headed self-attention matrix (MH-SA): **R**. This is done by first splitting up the MH-SA into the number of heads h and then concatenate these again. Hence the arrows at decoder. **R** is passed over to decoder.
- Decoder first uses next-word prediction from it's own MH-SA. This is the **Q** for the next step.
- This output (**Q**) of the masked MH-SA is disambiguated against the output of the encoder **R**, which is split into a **K** and **V**. See also previous slides with Matrix calculations “Je vais bien”.

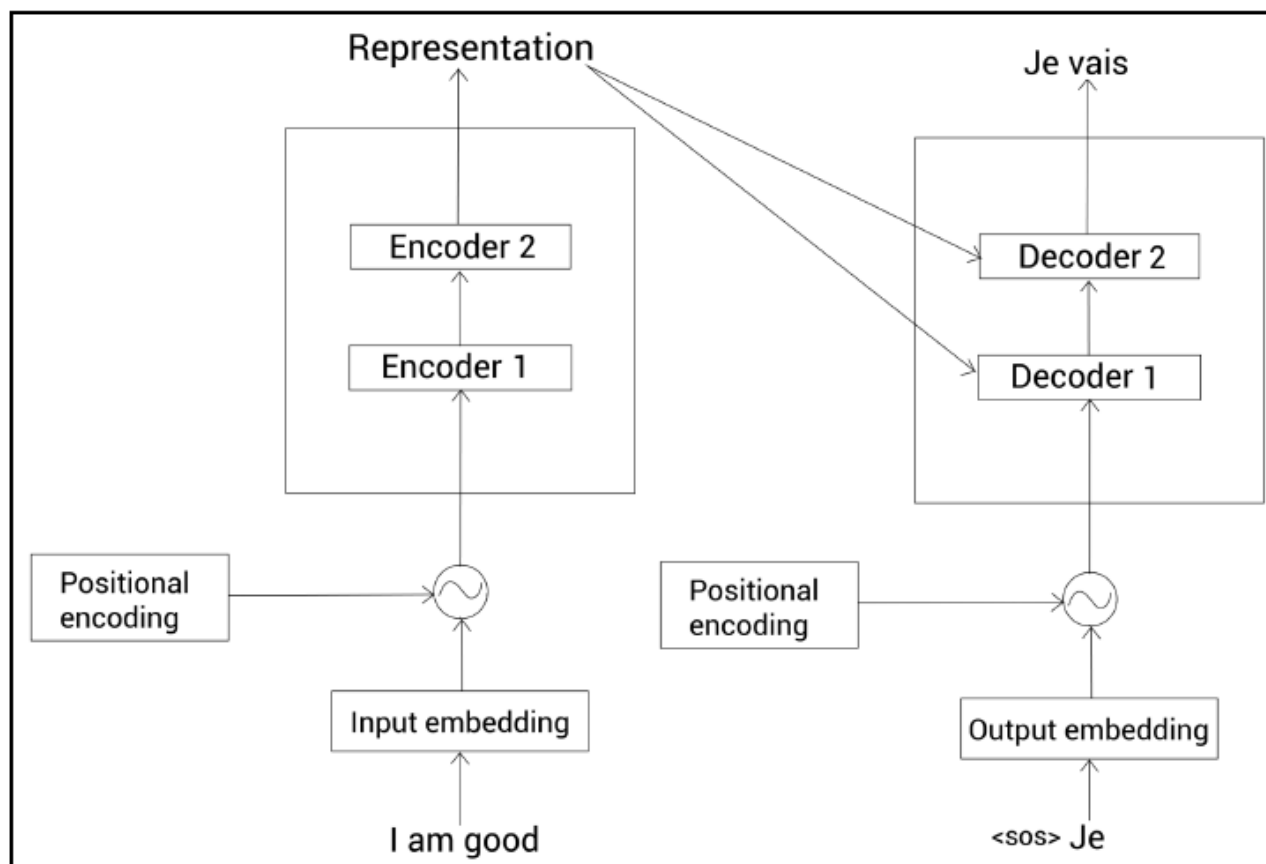


Decoder Mechanism

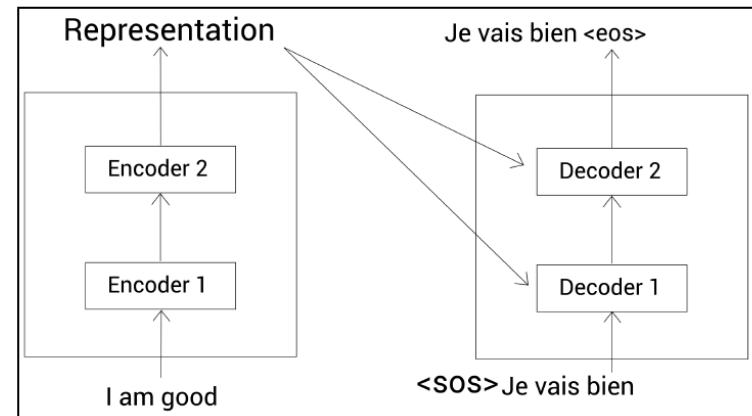
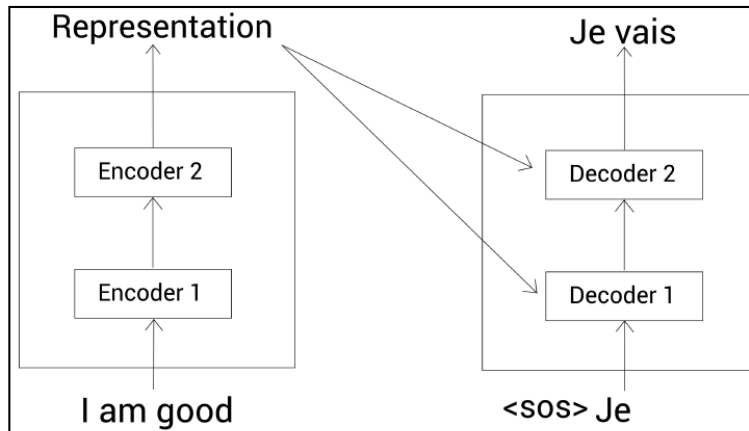
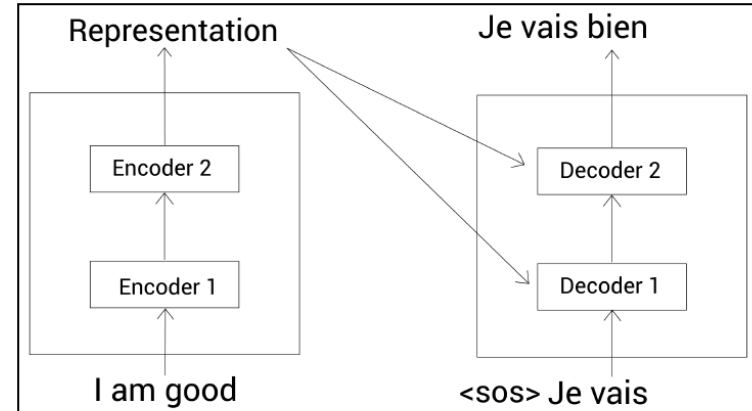
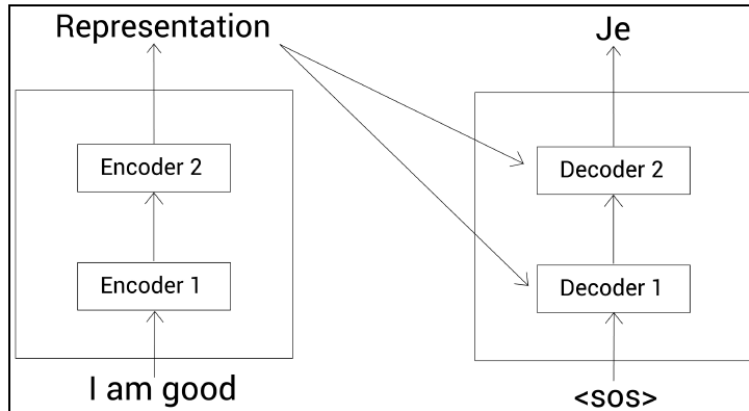
- The Transformer's layers of the decoder are (like the encoder) also a stack of layers
- The structure of the sub-layers is like the encoders. The output embedding layer and positional encoding are also the same as that of the encoder.
- Let's take a closer look...



Step $t=1$: input to the decoder will be $\langle \text{SOS} \rangle$ (start of sentence).
Decoder will generate word “Je”
Step $t = 2$: decoder generates $\langle \text{SOS} \rangle$ Je and from $t-1$ to generate
best next word: “vais”
Etc.



Decoding is (expensive) Iteration



- <sos>
- <sos> Je
- <sos> Je vais
- <sos> Je vais bien
- <sos> Je vais bien <eos>

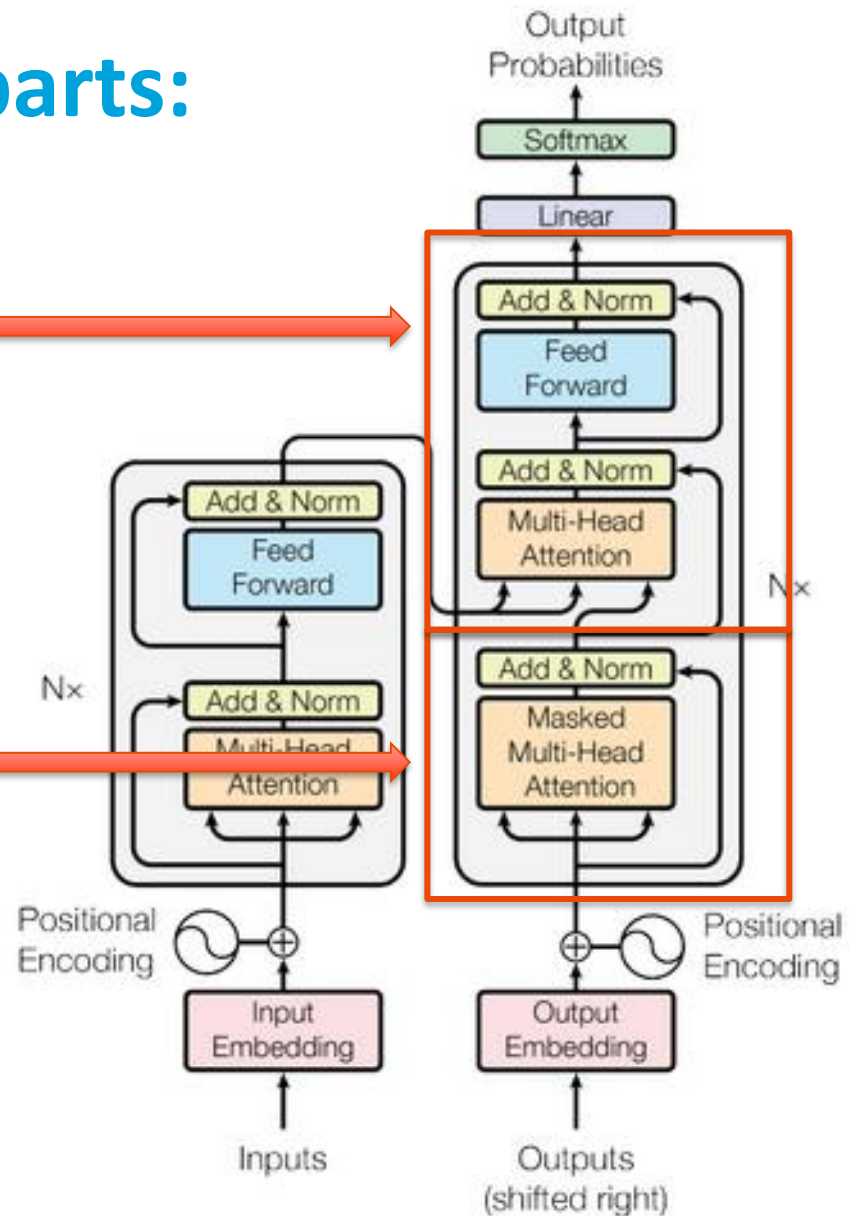


First, try "je", results in "vais".
Next try "je vais", results in
"bien", etc.

Decoding has two parts:

Decoding target sentence using multi-headed self-attention

Next Word Prediction (requires masking)



Masked Language Modeling for Learning

- “it is not possible to train bidirectional models by simply conditioning each word on its previous and next words, since this would allow the word that’s being predicted to indirectly “see itself” in a multi-layer model. To solve this problem, we use the straightforward technique of masking out some of the words in the input and then condition each word bidirectionally to predict the masked words.”

most	up	grow	necks	Africa
feet	leaves	bus	long	tallest



Based on the Cloze procedure: a technique for omitting words from a passage so that the reader is forced to use background experience, knowledge of syntax, vocabulary, interest, and, generally, higher order thinking skills to fill in the blank and complete the thought. (See also: “Cloze Procedure: A New tool for Measuring Readability”, 1953 W.L. Taylor)

Giraffes are the _____ living animal in the world. They can _____ up to about 5 metres tall. That is about as tall as a doubledecker _____!

Giraffes live in _____. Their long _____ help them to eat the _____ in the tallest part of the trees. They like the leaves on the acacia trees _____ of all.

Giraffes can run very fast but not for very _____. They can sleep standing _____ but often sleep sitting down with their _____ tucked under them.

Masking for Decoding

X =	<sos>	7.9	3.5	...	16.1	x_1
	Je	8.1	4.4	...	83.1	x_2
	vais	17	0.54	...	6.12	x_3
	bien	11.12	11.12...	22.1		x_4

<sos>	mask	mask	mask
<sos>	Je	mask	mask
<sos>	Je	vais	mask
<sos>	Je	vais	bien

- Masking is needed during training to prevent the attention mechanism of a transformer from “remembering” in the decoder during training.
- There is no need for this in the encoding in the basic transformer.
- We will get back to the need for masking during encoding of the BERT model (as this is due to the bi-directionality).

	<sos>	Je	vais	bien
<sos>	73	60	10	45
Je	40	99	25	70
vais	58	40	83	10
bien	12	11	15	80

$Q_i K_i^T =$

	<sos>	Je	vais	bien
<sos>	9.125	$-\infty$	$-\infty$	$-\infty$
Je	5.0	12.37	3.12	8.75
vais	7.25	5.0	10.37	1.25
bien	1.5	1.37	1.87	10.0

$\sqrt{Q_i K_i^T} = d_k$

	<sos>	Je	vais	bien
<sos>	9.125	7.5	1.25	5.625
Je	5.0	12.37	3.12	8.75
vais	7.25	5.0	10.37	1.25
bien	1.5	1.37	1.87	10.0

$\sqrt{Q_i K_i^T} = d_k$

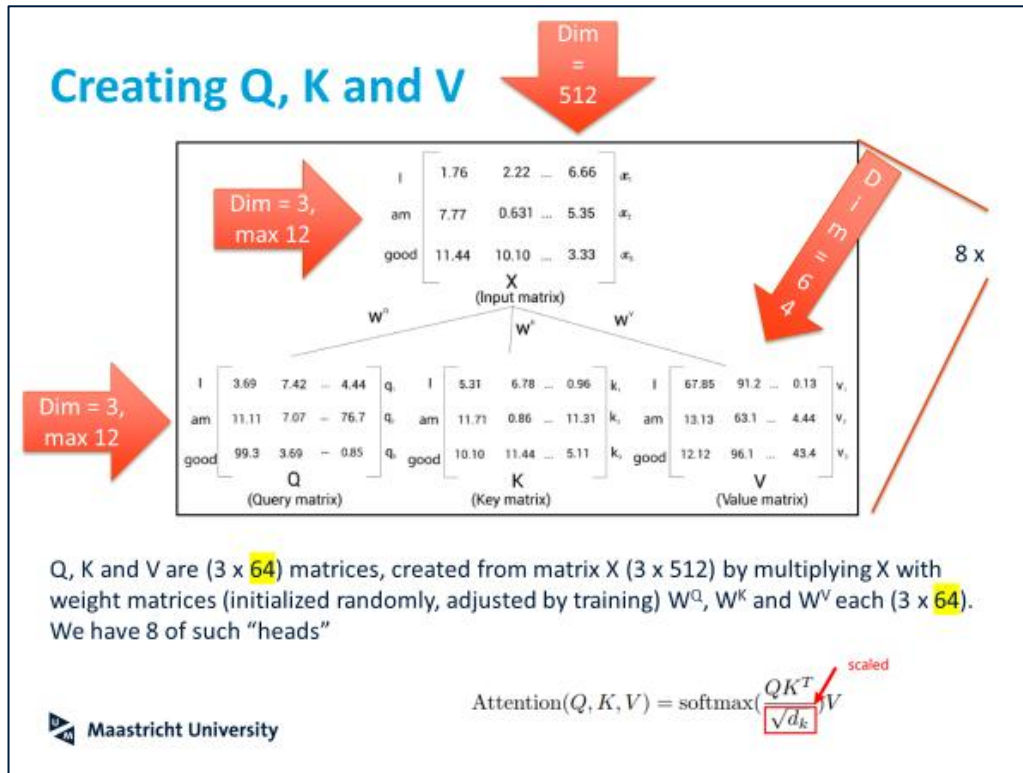
	<sos>	Je	vais	bien
<sos>	9.125	$-\infty$	$-\infty$	$-\infty$
Je	5.0	12.37	$-\infty$	$-\infty$
vais	7.25	5.0	10.37	1.25
bien	1.5	1.37	1.87	10.0

$\sqrt{Q_i K_i^T} = d_k$

	<sos>	Je	vais	bien
<sos>	9.125	$-\infty$	$-\infty$	$-\infty$
Je	5.0	12.37	$-\infty$	$-\infty$
vais	7.25	5.0	10.37	$-\infty$
bien	1.5	1.37	1.87	10.0

$\sqrt{Q_i K_i^T} = d_k$

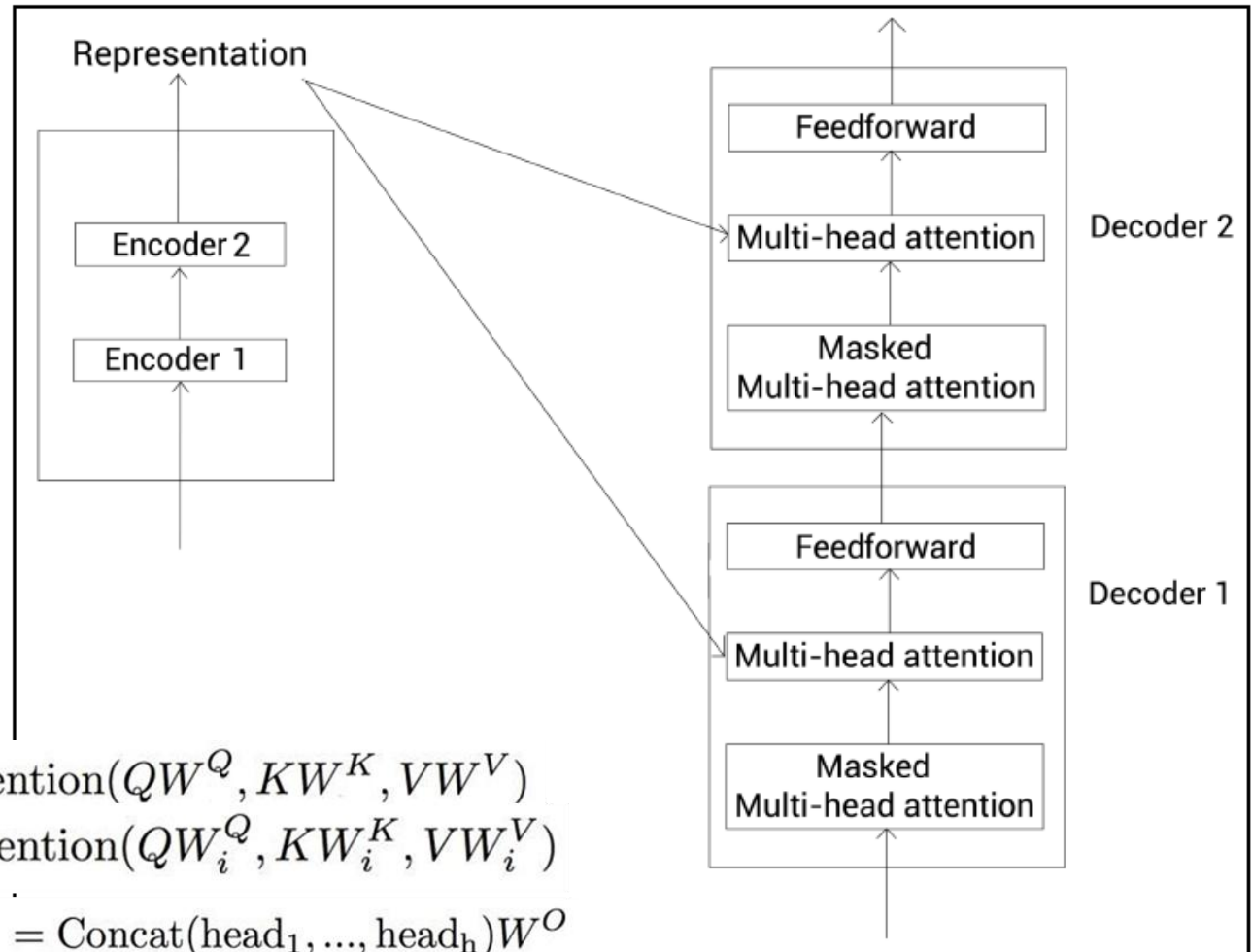
But we do not have only one head, we have 8



Self-Attention is a transformation to a latent space that allows us to understand (linguistic) relations better

Multi-headed Self-Attention allows us to extract information from different representation subspaces, which would, otherwise, not be possible with a single attention head. This results in better disambiguation and alignment of linguistic structures.

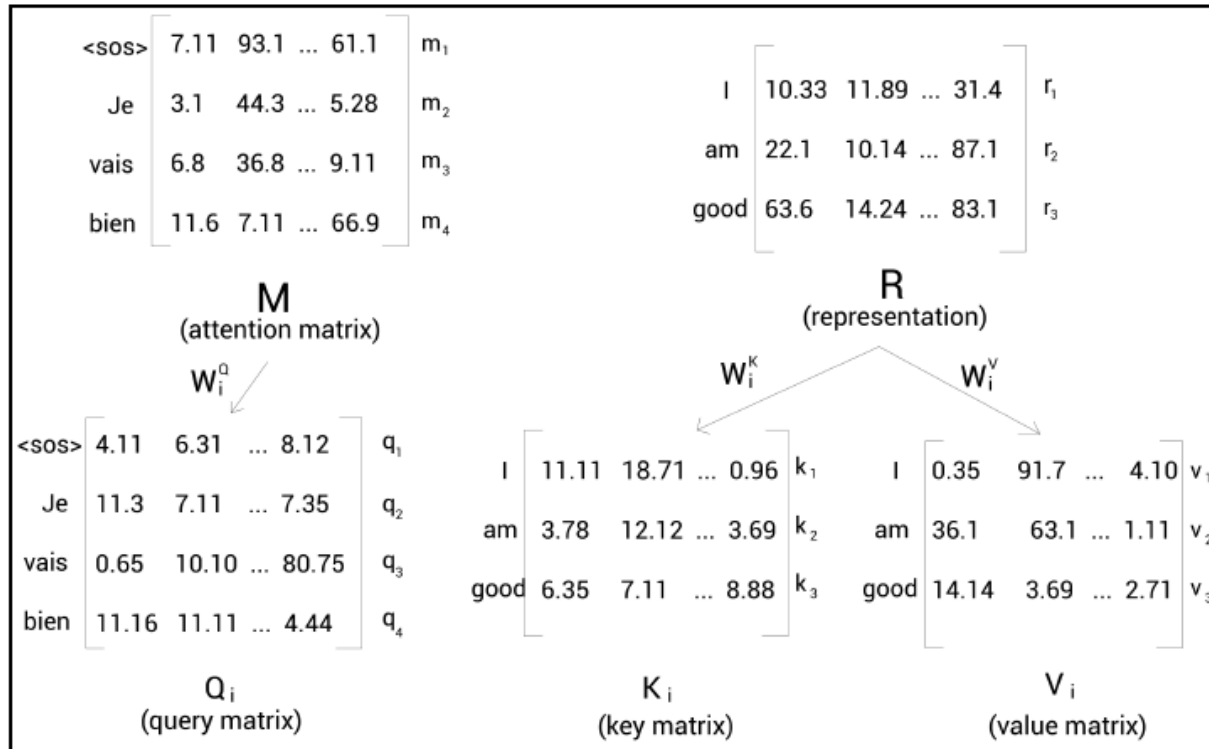
Now up to Multi-Headed (Cross) Attention Decoding



$$\text{head} = \text{Attention}(QW^Q, KW^K, VW^V)$$

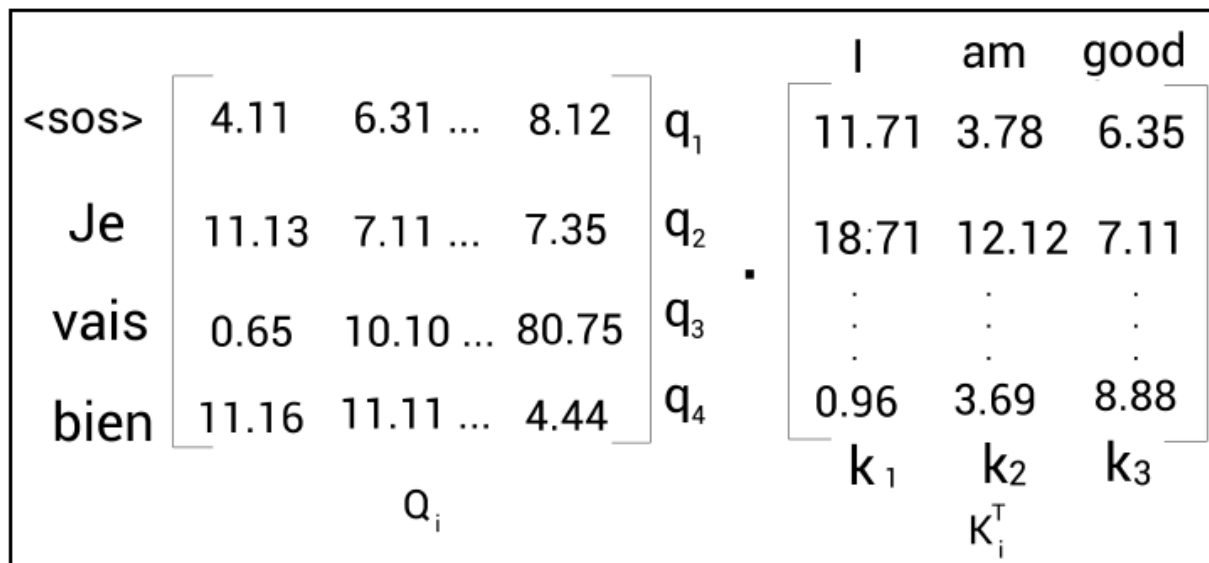
$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$



Decoding (cross)
attention:

- R is the encoder representation matrix
- M is the attention matrix from the previous sub-layer (decoded sequence).
- The head is represented by i
- W represents the weights set at training for Q, K and V for head i



- Now, we create Q_i by multiplying M and W_i^Q
- K and V are created by multiplying R with W_i^K and W_i^V
- Next we can multiply Q_i and K_i^T

$$\begin{array}{c}
 \begin{matrix} & & & & & \text{I} & \text{am} & \text{good} \\
 \begin{bmatrix}
 \text{<sos>} & 4.11 & 6.31 & \dots & 8.12 \\
 \text{Je} & 11.13 & 7.11 & \dots & 7.35 \\
 \text{vais} & 0.65 & 10.10 & \dots & 80.75 \\
 \text{bien} & 11.16 & 11.11 & \dots & 4.44
 \end{bmatrix} & \begin{matrix} Q_1 \\ Q_2 \\ Q_3 \\ Q_4 \end{matrix} \\
 Q_i
 \end{matrix}
 \cdot
 \begin{matrix}
 & & & & & \text{I} & \text{am} & \text{good} \\
 \begin{bmatrix}
 11.71 & 3.78 & 6.35 \\
 16.71 & 12.12 & 7.11 \\
 \vdots & \vdots & \vdots \\
 0.96 & 3.69 & 8.88
 \end{bmatrix} \\
 \begin{matrix} k_1 \\ k_2 \\ k_3 \end{matrix} \\
 K_i^T
 \end{matrix}
 =
 \begin{matrix}
 & & & & & \text{I} & \text{am} & \text{good} \\
 \begin{bmatrix}
 \text{<sos>} & q_1 k_1 & q_1 k_2 & q_1 k_3 \\
 \text{Je} & q_2 k_1 & q_2 k_2 & q_2 k_3 \\
 \text{vais} & q_3 k_1 & q_3 k_2 & q_3 k_3 \\
 \text{bien} & q_4 k_1 & q_4 k_2 & q_4 k_3
 \end{bmatrix} \\
 Q_i K_i^T =
 \end{matrix}
 \begin{matrix}
 & & & & & \text{I} & \text{am} & \text{good} \\
 \begin{bmatrix}
 \text{<sos>} & 91 & 60 & 77 \\
 \text{Je} & 96 & 63 & 12 \\
 \text{vais} & 41 & 111 & 48 \\
 \text{bien} & 36 & 45 & 65
 \end{bmatrix}
 \end{matrix}
 \end{array}$$

- Normalize it with $\text{Sqrt}(d_k)$
- Take SoftMax
- Multiply with V
- So at the end of the day, we take all context and into consideration.

$$\begin{array}{c}
 \begin{matrix} & & & & & \text{I} & \text{am} & \text{good} \\
 \begin{bmatrix}
 \text{<sos>} & 0.84 & 0.017 & 0.14 \\
 \text{Je} & 0.98 & 0.02 & 0.0 \\
 \text{vais} & 0.0 & 1.0 & 0.0 \\
 \text{bien} & 0.0 & 0.0 & 1.0
 \end{bmatrix} \\
 Z_i =
 \end{matrix}
 \begin{matrix}
 \text{softmax} \left(\frac{Q_i K_i^T}{\sqrt{d_k}} \right) \\
 V_i
 \end{matrix}
 \begin{matrix}
 & & & & & \text{I} & \text{am} & \text{good} \\
 \begin{bmatrix}
 0.35 & 91.7 & \dots & 4.10 \\
 36.1 & 63.1 & \dots & 1.11 \\
 14.14 & 3.69 & \dots & 2.71
 \end{bmatrix} \\
 \begin{matrix} V_i \end{matrix}
 \end{matrix}
 \end{array}$$

Sum=1

- We compare everything to everything
- In parallel with all heads.

Now take the highest values in Z

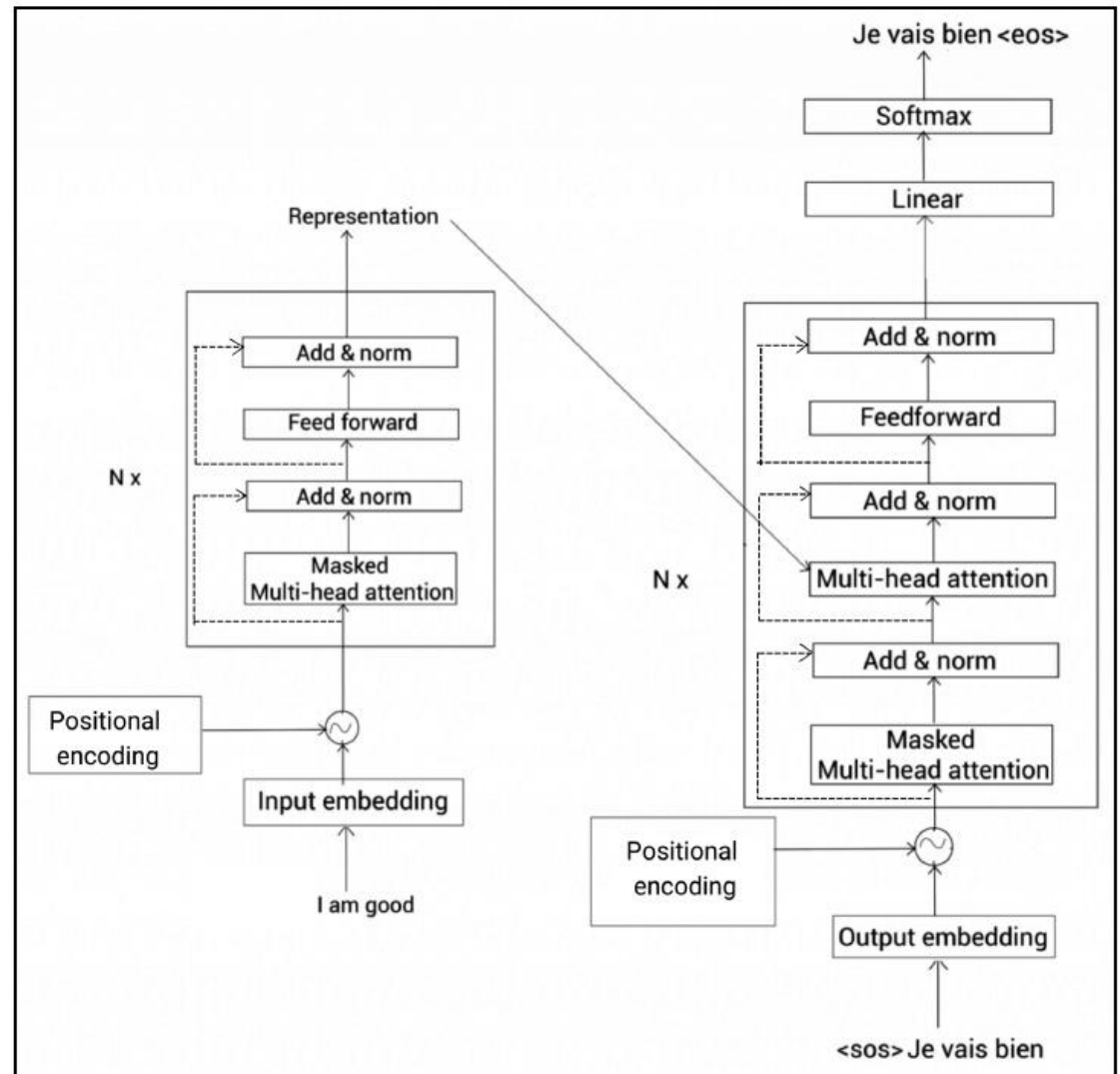
- Which will result in the best translation of “I am good” as “Je vais bien”.

$$Z_i = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ z_4 \end{bmatrix} \begin{matrix} <\text{sos}> \\ \text{Je} \\ \text{vais} \\ \text{bien} \end{matrix}$$

$z_2 = 0.98$	0.35	91.7	...	+0.02	36.1	63.1	...	+ 0.0	14.14	3.69	...
	$V_1(l)$				$V_2(am)$				$V_3(good)$		

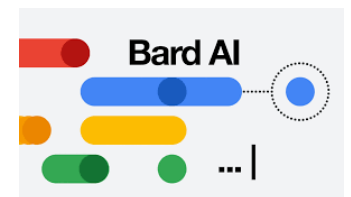
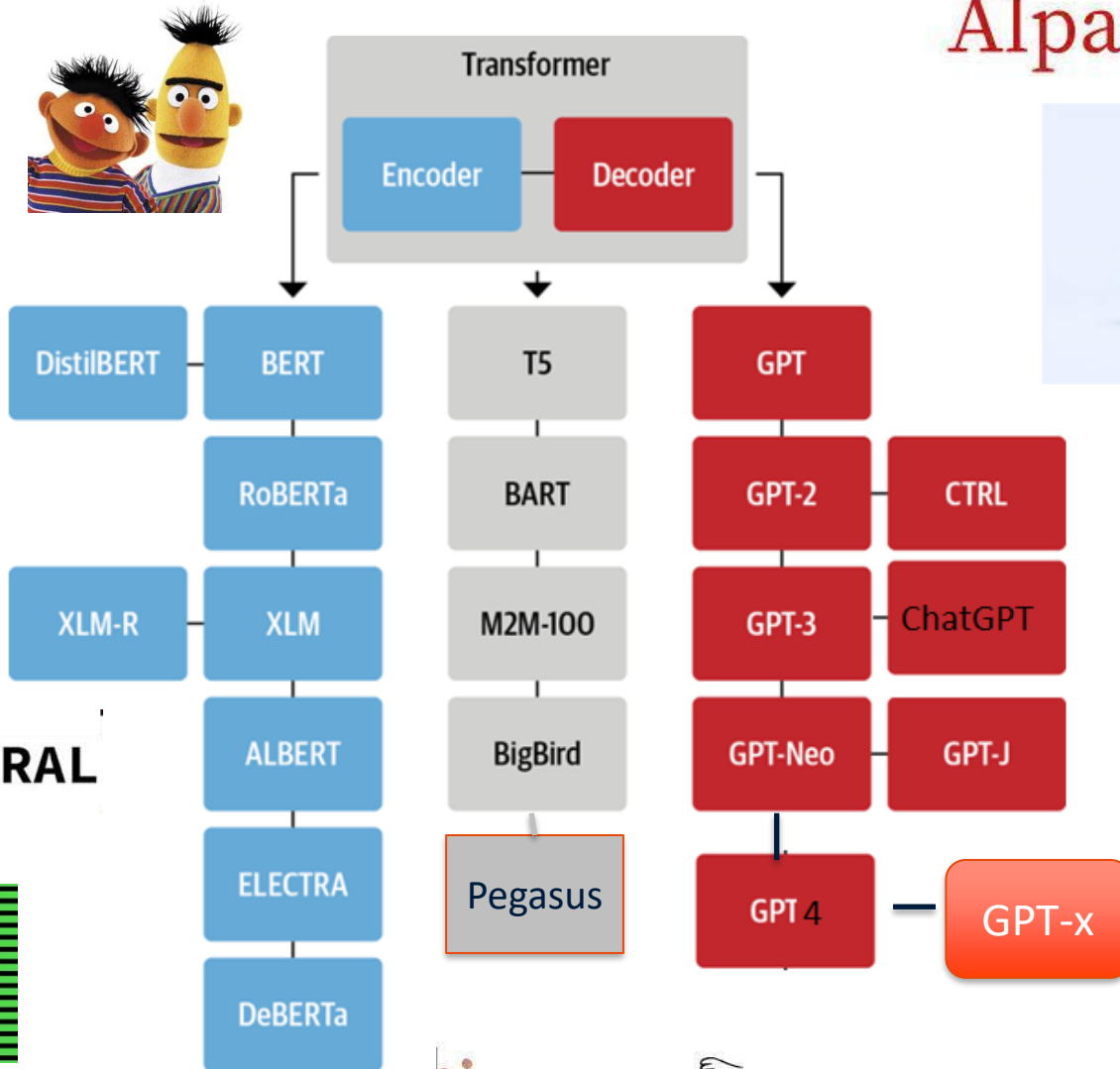
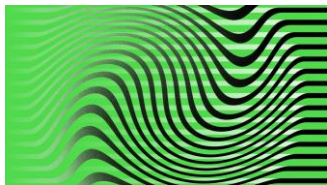
Et voila!

- Transformers can uncover mind-blowing long-distance dependencies in natural language.
- Transformers can perform transductions from oral and written sequences to meaningful representations never seen before in the history of NLP.
- Removing recurrent network layers in transformers, allows for parallelized layers that reduce training time



The Transformer Zoo

Stanford
Alpaca



Now let's form groups of 4



Discuss Exercises Lecture

Exercise 1: Scaled Dot-Product Mechanics and Multi-Head Dimensions

In the Transformer architecture (*Attention Is All You Need*), multi-head self-attention linearly projects input representations into multiple representation subspaces to compute attention in parallel.

Assume an input sequence of length $n = 16$ tokens processed with the standard base configuration: model embedding dimension $d_{\text{model}} = 512$ and $h = 8$ attention heads.

- Determine the dimensionality per head (d_k and d_v). For a single attention head i , state the exact matrix dimensions of the query projection weight W_i^Q , the query matrix Q , the key matrix K , and the transposed key matrix K^T .
- Write down the scaled dot-product attention formula:
- $$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$
- What is the matrix shape of the raw attention logits QK^T ? Along which axis is the softmax applied, and why is scaling by $\frac{1}{\sqrt{d_k}}$ necessary when computing dot products in high dimensions?
- Show the dimensions of the single-head output matrix $Z_i = \text{Attention}(Q_i, K_i, V_i)$. Once all $h = 8$ heads are concatenated ($\text{Concat}(\text{head}_1, \dots, \text{head}_h)$), what is the resulting dimension, and what are the dimensions of the final linear projection matrix W^O needed to project the multi-head output back to d_{model} ?

Exercise 2: Sinusoidal Positional Encoding Derivation and Properties

Because Transformers process all tokens in parallel rather than sequentially through recurrent loops, the architecture lacks inherent awareness of token order. To inject sequence information, fixed sinusoidal positional encodings (PE) are added element-wise to the input token embeddings.

- $PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$
- Calculate the exact values for the first four dimensions ($2i = 0, 2i + 1 = 1, 2i = 2, 2i + 1 = 3$) for token position $pos = 0$. What repeating numerical pattern appears across the entire vector for $pos = 0$?
- The paper hypothesizes that this formulation enables the model to easily learn relative positions because for any fixed offset k , PE_{pos+k} can be represented as a linear transformation of PE_{pos} . Using the trigonometric addition theorems for $\sin(\alpha + \beta)$ and $\cos(\alpha + \beta)$, describe how a 2×2 rotation matrix $\Phi_m^{(k)}$ maps the pair $(\sin(\lambda_m \cdot pos), \cos(\lambda_m \cdot pos))$ to $(\sin(\lambda_m(pos + k)), \cos(\lambda_m(pos + k)))$ independently of the absolute position pos .
- Why are residual connections ($x + \text{Sublayer}(x)$) essential after each attention block to ensure that these positional signals are not lost or overwritten as representations pass through deep stacks ($N \times$) of encoder and decoder layers?

Exercise 3: Encoder-Decoder Cross-Attention and Computational Bottlenecks

Review the structural flow of the complete Transformer encoder-decoder model and its computational complexity.

- In the decoder block, distinguish between **Masked Multi-Head Attention** and **Encoder-Decoder Cross-Attention**:
 - Why is causal masking required in the decoder's self-attention during autoregressive training?
 - In cross-attention, which sublayer provides the queries (Q), and which sublayer provides the keys (K) and values (V)? Explain how this routes information from the source sentence to guide target next-word generation.
- Explain the structural purpose of the position-wise Feed-Forward Network (FFN) placed after attention in each layer. Why would stacking multiple self-attention layers without non-linear feed-forward layers collapse into a series of linear re-averaging operations?
- Consider the computational complexity of a Transformer layer: self-attention scales as $O(n^2d)$, while the feed-forward network scales as $O(nd^2)$ (where n is sequence length and d is model dimension). When scaling models to long document contexts (e.g., $n > 8,000$), explain why the self-attention operation—and its corresponding attention weight memory $O(n^2)$ —becomes the primary computational and memory bottleneck on modern hardware.

Discussion



On the Hardware Lottery: Did Attention Win, or Did the GPU?

In 2017 “Attention Is All You Need” discarded recurrence and built a model from nothing but matrix products. It outperformed LSTMs. It also happened to be exactly the computation that GPUs and TPUs had been built to do fastest.

Did the Transformer win because self-attention is the right mathematical structure for language, or because it was the first NLP architecture that matched the silicon? (Hooker calls this the hardware lottery.)

If the winning architecture is the one that fits the chip, then our best model of language is also a portrait of our hardware. Is there any experiment that could tell the two apart?

On Positional Encoding: Is Time Essential to Language, or Only to Us?

Language happens in time: a sentence is heard one word after another, and “Jane hits John” is not “John hits Jane”. Self-attention has no time. It sees the whole sentence at once, as a set, and without positional encodings it cannot tell the two sentences apart.

Order is then added back as a signal: sinusoidal waves mixed into the embeddings. The architecture treats time as one more feature of space.

Is a system that has to be told about order processing language, or processing a picture of language? And if the picture works better than the sequence ever did, was order essential to language, or only to how humans happen to receive it?



On Forgetting: Is Perfect Memory the Goal, or the Problem?

Self-attention compares every token with every other token and never forgets anything inside its window. A human reading a 500-page novel forgets almost every sentence and understands the book anyway.

Is bounded memory a limitation that biology was stuck with, or is forgetting part of how understanding works: keeping what matters by letting go of what does not?

If the quadratic cost of attention is the price of never choosing what to forget, is the long-context race solving the right problem? What would a model that forgets well look like, and how would we know it was understanding rather than losing?

Questions?



Maastricht University